



Realtime Video Orchestration Technical Documentation

[Architecture](#) • [Implementation Reference](#) • [Developer Guide](#)
[Low-Latency Systems Primer](#) • [Benchmark Suite](#) • [Performance Results](#)

[Complete Technical Reference](#)

[Rust • 15 Crates](#)

Generated June 06, 2026

RVO: Realtime Video Orchestration — Technical Documentation

- [RVO: Realtime Video Orchestration](#)
 - [Why RVO Exists](#)
 - [What It Does](#)
 - [Core Principles](#)
 - [Measured Performance](#)
 - [Current State](#)
 - [Quick Start](#)
 - [Documentation](#)
- [Architecture](#)
 - [System Invariant](#)
 - [End-to-End Data Flow](#)
 - [Bounded Everything](#)
 - [Frame Buffer Sharing](#)
 - [Scheduler and DetectorNode Contract](#)
 - [Remote Detector Isolation](#)
 - [Signal Store](#)
 - [Condition DSL](#)
 - [Event Engine](#)
 - [Evidence Pipeline](#)
 - [Load Shedding](#)
 - [Front-Ends](#)
 - [Observability](#)
 - [Design Boundaries](#)
 - [Scaling](#)
- [Current Implementation](#)
 - [Repository Shape](#)
 - [Entrypoint](#)
 - [Configuration](#)
 - [Camera Path](#)
 - [Frame Buffer](#)
 - [Detector Model](#)
 - [Scheduler](#)
 - [Signal Store](#)
 - [Condition DSL](#)
 - [Event Engine](#)
 - [Evidence Pipeline](#)
 - [Remote Detectors](#)
 - [Event Publishers](#)
 - [Metrics](#)
 - [Hot Reload](#)
 - [Build and Run](#)
- [RVO Developer Guide](#)
 - [1. Why Rust](#)
 - [2. What RVO Is](#)
 - [3. Crate Layout](#)
 - [4. End-to-End Data Flow](#)
 - [5. Key Design Decisions and Tradeoffs](#)
 - [6. Known Caveats and Open Problems](#)

- 7. Running RVO: the CLI and the TUI
- 8. Practical Dev Workflow
- 9. Repo Structure
- Low-Latency Systems — Concepts, Terms, and How RVO Applies Them
 - 0. The one idea everything hangs off
 - 1. Latency vs Throughput
 - 2. Tail latency (the heart of the field)
 - 3. The fast-path / slow-path split
 - 4. Queueing theory you actually need
 - 5. Backpressure
 - 6. Load shedding & admission control
 - 7. Freshness, staleness, and TTL
 - 8. Determinism and the runtime (why language/runtime choice matters)
 - 9. Concurrency models
 - 10. Memory discipline
 - 11. The control-loop pattern
 - 12. Reliability under load & failure isolation
 - 13. Observability
 - 14. Capacity planning & scaling
 - 15. Networking for low latency
 - 16. Common anti-patterns (what *not* to do)
 - 17. How RVO embodies the domain (one-screen map)
 - Glossary
- RVO Benchmarking Guide
 - Table of Contents
 - 1. Why benchmarking matters here
 - 2. Suite overview
 - 3. Environment requirements
 - 4. Running the micro-benchmarks
 - 5. Running the macro load harness
 - 6. Output files — schema and meaning
 - 7. Generating figures
 - 8. Interpreting each figure
 - 9. Statistical rigour checklist
 - 10. Adding a new scenario
 - 11. Common pitfalls
 - 12. Using the numbers in the tech report
- RVO Benchmark Performance
 - 1. Methodology
 - 2. Measured Results
 - 3. Architectural Claims Confirmed
 - 4. Limitations
- Benchmark Plot Guide
 - Running the bench suite
 - CSV schema
 - The five figures
 - Plotting recipe (any tool)
 - Interpreting validation output

RVO: Realtime Video Orchestration

RVO is low-latency scheduling infrastructure for realtime video AI. It ingests live streams, schedules models under latency budgets, routes results through bounded message paths, fuses outputs into temporal events, and extracts evidence clips — without the live path ever waiting for slow work.

Why RVO Exists

Modern video AI systems are not limited only by model accuracy. In production, they are often limited by orchestration:

- Frame backlogs create stale inference.
- Slow models block fast models.
- Unbounded queues hide latency until the system falls behind.
- Ad hoc glue code makes detector dependencies hard to reason about.
- One-frame detections create noisy events.
- Clip extraction and storage compete with realtime inference.
- Production teams lack clear metrics for latency, drops, and overload.

RVO treats realtime video AI as a data systems problem. Instead of asking every application team to reinvent capture loops, model schedulers, signal freshness, event debouncing, and evidence capture, RVO provides a common runtime for those concerns.

What It Does

Realtime Frame Bus	bounded camera ingestion, circular frame buffer
Model Orchestrator	FPS-gated, cost-aware, dependency-chained scheduler
Signal Store	typed, TTL-bounded blackboard between models
Temporal Event Engine	condition DSL, Idle→Potential→Confirmed state machine
Evidence Pipeline	post-roll clip capture, JPEG+metadata, best-effort
Remote Model Gateway	non-blocking gRPC fan-out to external model services
Observability Layer	Prometheus counters, /health, load harness with HDR histograms

Core Principles

Bounded Everything — Frame buffers, channels, job queues, and signal state have fixed bounds. RVO does not hide overload behind unbounded memory growth.

No Stale Work — Old frames are less valuable than fresh frames. If the system falls behind, RVO drops stale work instead of processing it late.

Time-Aware Scheduling — Models run according to time budgets, FPS caps, dependency freshness, and load policy. The goal is not to run every model on every frame; it is to run the right model at the right time with bounded latency.

Signal-Oriented Modularity — Models emit typed signals that other modules consume if fresh. Models do not call each other directly.

Temporal Correctness — Events are confirmed over time, not triggered by every raw detection. A sustained condition is more useful than a noisy spike.

Failure Isolation — Slow encoding, disk writes, or downstream consumers never block capture or scheduling. Evidence is best-effort.

Production Observability — A realtime runtime proves its behavior with metrics: frame drops, model latency, scheduler ticks, event counts, clip pipeline health, and queue pressure.

Measured Performance

5 runs × 30 s each on bare-metal Linux, performance governor. Median across runs.

Scenario	tick_p50	tick_p99	frame_loss_rate
Baseline (no detectors)	4.8 μ s	10.8 μ s	0
HOL blocking — 10 ms detector, every tick	10.16 ms	10.34 ms	0
Load-shedding — 50 ms detector shed by backoff	4.9 μ s	13 μ s	0
Overload — 577 fps camera, 5 ms detector	5.08 ms	5.21 ms	401/s
Throughput ceiling — 3841 fps actual (DummyDetector)	5.8 μ s	13.7 μ s	2087/s

HOL blocking appears directly in tick p50. Load-shedding keeps p50 at baseline despite a 50 ms detector. Under overload, p99 stays bounded while ring-buffer loss absorbs excess frames. See [BENCHMARK_PERFORMANCE.md](#) for full results, methodology, and all architectural claims confirmed by measurement.

Current State

Working Rust implementation, 15 crates, CI passing.

Core runtime - OpenCV camera capture with RTSP/URI support - Bounded frame channel (cap 5) + circular frame buffer (cap 300, ~10 s at 30 fps) - Time-gated, cost-aware, dependency-chained scheduler with HOL backoff (Medium 100 ms / High 500 ms) - Typed signal store (Dummy, MotionLevel, FacePresent, PersonDetected) with TTL freshness - Composable condition DSL (`ALL` / `Any` over typed signal predicates) - Temporal event engine (Idle $\rightarrow$ Potential $\rightarrow$ Confirmed $\rightarrow$ Cooldown) - Post-roll clip capture — JPEG frames + JSON metadata sidecar, best-effort, bounded thread

External model integration - `RemoteGrpcDetector` — non-blocking gRPC fan-out to any model service speaking the `rvo.detect.v1.Detector` proto - Persistent HTTP/2 channels, lazy dial, TTL-bounded cached results - Failure threshold $\rightarrow$ `DetectorHealth::Failed` $\rightarrow$ scheduler disables detector, pipeline continues

Front-ends - `rvo` CLI — headless runtime; `--detector ENDPOINT=SIGNAL`, `--camera-device`, `--list-cameras` - `rvo-tui` — interactive terminal dashboard: signals, metrics, events - `rvo-web` — browser dashboard with live node-add API at `POST /api/nodes`

Testing and benchmarking - `rvo-testkit` — synthetic cameras, scripted / latency / failing / chained detectors, full pipeline builder - `rvo-scenarios` — 12 end-to-end integration scenarios - `rvo-bench` — load harness: 13 scenarios, HDR latency histograms, multi-run CSV output - CI: check, test, clippy, fmt across all 15 crates

See [CURRENT_IMPLEMENTATION.md](#) for the precise behavior of the running code.

Quick Start

```
# Requires: Rust stable, OpenCV system libraries
cargo build --release -p rvo-bin

# Discover cameras
./target/release/rvo --list-cameras

# Run headless
RVO_CONFIG=config/rvo.yaml ./target/release/rvo

# Interactive TUI
cargo run -p rvo-bin --bin rvo-tui

# Browser dashboard (open http://127.0.0.1:8080)
cargo run -p rvo-bin --bin rvo-web

# Point at an external model service
./target/release/rvo --detector http://localhost:50051=PersonDetected

# Run benchmarks (5 runs x 30 s each → 65-row CSV)
cargo build --release -p rvo-bench --bin load_harness
./target/release/load_harness --all --runs 5

# Observe
curl http://127.0.0.1:9090/metrics
curl http://127.0.0.1:9090/health
tail -f events.jsonl
ls clips/
```

Documentation

Document	Purpose
ARCHITECTURE.md	Data flow, component contracts, design boundaries
DEVELOPER_GUIDE.md	Language choice, crate layout, design decisions and tradeoffs
CURRENT_IMPLEMENTATION.md	What the running code does today
BENCHMARKING.md	How to run the benchmark suite, scenario definitions, CSV schema
BENCHMARK_PERFORMANCE.md	Methodology rationale, measured results, architectural claims
PLOT_GUIDE.md	Figure descriptions, axes, plotting recipes
LOW_LATENCY_SYSTEMS.md	Domain primer: latency, queuing, scheduling, observability

Architecture

This document describes the RVO system design: the invariant it enforces, the data flow, the component contracts, and the design decisions that follow from them. For implementation specifics (crate layout, type signatures, exact behavior), see [CURRENT_IMPLEMENTATION.md](#).

System Invariant

The live path must never wait for slow work.

The live path: camera ingestion → frame buffer → scheduler → detector execution → signal publication → event evaluation. Everything else (encoding, file I/O, downstream delivery, network calls) is off the live path.

End-to-End Data Flow

```

Camera / RTSP Stream
|
v (bounded channel, cap 5 – drops on full)
Scheduler.tick()
|
+--> FrameBuffer (Arc<Mutex<>>, circular, cap 300)
|
+--> DetectorNode(s)
|
|   |
|   |   In-process detector
|   |   execute() runs inline, returns signals
|   |
|   |   Remote gRPC detector (rvo-remote)
|   |   execute() is NON-BLOCKING:
|   |   • writes newest frame to single-slot mailbox
|   |   • returns cached signals (TTL-stamped)
|   |   worker thread (own Tokio rt, persistent
|   |   HTTP/2 channel) ↔ external model service
|   |
|   v (both paths write to SignalStore)
|   SignalStore (typed slots, TTL freshness)
|
+--> EventEngine (Condition DSL: All/Any over SignalPredicates)
|
|   v
|   Vec<Event>
|
+--> EventPublisher (try_send, cap 64 – drops on full)
|
|   +--> stdout logger
|   +--> JSON-lines file sink (optional)
|
+--> ClipManager (try_send to pending queue, cap 16 – drops on full)
|
|   v (single worker thread: sleeps until post-roll window closes)
|   (try_send to encoder, cap 8 – drops on full)
|   Encoder Worker
|   |
|   v
|   clips/{type}_{ts}/frame_NNNN.jpg + meta.json

```

Bounded Everything

Every handoff in the system uses a bounded structure:

Handoff	Bound	Overflow behavior
Camera → Scheduler	Channel cap 5	Drop frame, count <code>rvo_frame_drops_total</code>
Remote detector mailbox	1 slot (overwrite-newest)	Stale frame silently replaced
Scheduler → EventPublisher	Channel cap 64	Drop event, count <code>rvo_event_drops_total</code>
Scheduler → ClipManager (pending queue)	Channel cap 16	Drop clip job, count <code>rvo_clip_drops_total</code>
ClipManager worker → Encoder	Channel cap 8	Drop clip job, count <code>rvo_clip_drops_total</code>
FrameBuffer	300 frames (~10 s @ 30 fps)	Overwrite oldest

No unbounded queue exists anywhere in the live path.

Frame Buffer Sharing

The frame buffer is the only state shared between the live path and the evidence pipeline. It uses

`Arc<Mutex<FrameBuffer>>`:

- **Scheduler** locks briefly each tick to drain frames and snapshot the newest timestamp, then releases.
- **ClipManager** locks briefly on event to read `newest_instant()` (anchors the clip window), then releases. A single long-lived worker sleeps until the post-roll window closes, then locks briefly to slice frames. The lock is never held while sleeping.

The contention window is one `slice()` call, not the entire post-roll duration.

Scheduler and DetectorNode Contract

The scheduler is a clock-driven arbiter. It decides when each detector may run.

Scheduler responsibilities: enforce FPS caps, check signal dependency freshness, apply load-shedding backoff, measure execution latency, handle `Failed` health.

DetectorNode responsibilities: consume the provided context (timestamp, optional frame, signal store), produce typed signals and a health status, own no scheduling, I/O, or downstream delivery logic.

Neither touches the other's domain. This contract holds for both in-process and remote detectors — `RemoteGrpcDetector` satisfies `DetectorNode` while keeping the network call off the scheduler tick.

Remote Detector Isolation

The key design challenge for remote detectors: a network round-trip plus inference is the slowest thing in the system. A naive blocking `execute()` would either stall the tick or trip its own backoff on every frame.

The solution decouples inference from the tick:

scheduler tick (hot)	worker thread (persistent HTTP/2 channel)
<code>execute(ctx):</code>	<code>loop:</code>
write frame → mailbox →	take newest frame
read cached result ←	JPEG-encode + Detect() over gRPC
return signals (TTL)	store (value, produced_at)

`execute()` never waits on the network. Staleness is handled by the existing TTL mechanism: if the worker falls behind, the cached result expires in `SignalStore` exactly as an in-process signal would. Failure accumulates to a threshold, then `DetectorHealth::Failed` — the scheduler disables the detector and the pipeline continues.

Signal Store

The signal store is a typed blackboard, not a queue. It answers:

What is the latest valid value of signal X right now?

One slot per `SignalType`. Reads and writes are $O(1)$ by type index. Freshness is enforced at read time via TTL: `signal.ts_ns + signal.ttl_ns < now_ns` → absent.

A missing or stale signal is treated as absent — dependent detectors are skipped, predicates referencing it evaluate false.

Condition DSL

Event conditions are a tree of signal predicates:

```
Condition
├── All(predicates)    → AND semantics
└── Any(predicates)    → OR semantics

SignalPredicate
├── signal_type: SignalType
├── op: CompareOp (Gte | Gt | Eq | Lt | Lte)
└── value: u64
```

Each `EventMachine` evaluates its own `Condition` independently per tick.

Event Engine

Temporal state machine per event definition:


```

Idle → (condition first true) → Potential { start_ns }
      → (condition still true, elapsed ≥ duration_ns) → emit Event
      → Cooldown { until_ns }
      → (now ≥ until_ns) → Idle

(condition becomes false while Potential) → back to Idle

```

Event emission is separate from evidence capture. An event is meaningful even if clip extraction fails or is dropped.

Evidence Pipeline

The evidence pipeline is explicitly best-effort:

- Clip jobs use `try_send` — never blocks the live path.
- A single long-lived worker thread (not per-event threads) processes jobs sequentially, keeping thread count bounded regardless of event burst size.
- Failed or slow encoding does not affect the scheduler.

Load Shedding

When a detector's execution time exceeds 2× its FPS budget, it enters backoff:

```

cost_hint → backoff duration
Low       → 0 ms    (always recovers immediately)
Medium    → 100 ms
High      → 500 ms

```

During backoff the detector is skipped entirely — no missed executions are queued. RVO degrades by doing less work per unit time, not by accumulating stale work.

Front-Ends

All three front-ends run the same runtime:

- `rvo` — headless CLI for servers and scripted use.
- `rvo-tui` — interactive terminal dashboard: signal blackboard, metrics, events.
- `rvo-web` — browser dashboard; supports live model node addition via `POST /api/nodes` without restart.

Observability

Every significant state change produces a metric increment. Prometheus-style counters at `/metrics`; process-alive check at `/health`. Key families: frame drops, scheduler tick rate, detector exec/skip/latency, event counts, clip drops.

The load harness (`rvo-bench`) provides HDR histogram latency measurements (p50/p99/p99.9) for the scheduler and detector paths across 13 benchmark scenarios. See [BENCHMARK_PERFORMANCE.md](#) for measured results.

Design Boundaries

RVO is **not**: - a computer vision model or training framework - a video codec or muxer - a general-purpose distributed compute engine - a durable event log

RVO **is**: - a realtime orchestration runtime for video AI models - a bounded message path for frames and signals - a time-aware scheduler for model execution - a temporal event confirmation engine - a best-effort evidence pipeline

Scaling

Today: single process per camera stream. Remote detectors (`rvo-remote`) already move the model execution out-of-process, behind a stable gRPC contract. Each remote detector is a worker thread and a persistent channel — adding a model node is O(1) on the hot path.

Future: the component boundaries (ingest, scheduler, detector pools, event publisher, clip encoder, metrics) are chosen so each can move across a process or host boundary. The core contracts — bounded queues, freshness-

first scheduling, typed signals, temporal event semantics, no slow work on the live path — should hold in the distributed form as they do today.

Current Implementation

This document describes what the current Rust code does. It is kept separate from the product vision so implementation status stays unambiguous.

Repository Shape

Rust workspace with fifteen crates under `crates/`. All are listed as workspace members so `cargo test --workspace` covers every crate.

Crate	Responsibility
<code>rvo-bin</code>	Entrypoint, runtime wiring, SIGHUP reload; also <code>rvo-tui</code> and <code>rvo-web</code> binaries
<code>rvo-config</code>	YAML config loading and validation
<code>rvo-camera</code>	OpenCV capture, RTSP/URI source
<code>rvo-buffer</code>	Bounded circular frame buffer
<code>rvo-detector</code>	Detector trait, synthetic detectors
<code>rvo-remote</code>	<code>RemoteGrpcDetector</code> — non-blocking gRPC fan-out; persistent HTTP/2 channel per model node
<code>rvo-signals</code>	Typed signal store
<code>rvo-events</code>	Condition DSL, temporal event engine, publishers
<code>rvo-scheduler</code>	Orchestration loop, load shedding
<code>rvo-clips</code>	Clip job pipeline, JPEG encoder, JSON metadata
<code>rvo-metrics</code>	Prometheus-style counters, HTTP endpoints
<code>rvo-core</code>	Shared time helper
<code>rvo-testkit</code>	Test infrastructure: synthetic cameras, scripted/latency/failing detectors, pipeline builder
<code>rvo-scenarios</code>	End-to-end integration scenarios using <code>rvo-testkit</code>
<code>rvo-bench</code>	Load harness: 13 scenarios, HDR latency histograms, multi-run CSV output

Entrypoint

`rvo-bin` ships three binaries: - `rvo` (`src/main.rs`) — headless CLI - `rvo-tui` (`src/bin/rvo-tui.rs`) — interactive terminal dashboard - `rvo-web` (`src/bin/rvo-web.rs`) — browser dashboard with live node-add API

All three run the same runtime. `rvo` startup order:

1. Read config path from `RVO_CONFIG` env var (default: `config/rvo.yaml`).
2. Start metrics server on `127.0.0.1:9090`.
3. Load and validate config.
4. Create the shared `Arc<Mutex<FrameBuffer>>`.
5. Build detectors and event engine from config.
6. Start camera thread.
7. Start clip encoder worker thread.
8. Optionally start event file sink if `event_log` is set in config.
9. Build scheduler (receives the shared frame buffer).
10. Spawn SIGHUP reload thread (Unix only; no-op on Windows).

11. Enter the 1 ms tick loop.

Configuration

Active config: `config/rvo.yaml`.

Top-level fields:

- `camera`: source definition (`device_index` or `source_uri`).
- `detectors`: list of detector definitions.
- `events`: list of event definitions.
- `clips_dir`: output directory for evidence clips (default: `clips/`).
- `event_log`: optional path for JSON-lines event output (e.g. `events.jsonl`).

Detector kinds: `dummy`, `load`, `jitter`.

Event `condition` block supports `all` and `any` predicates. The shorthand `signal_type` + `signal_threshold` fields expand to a single `gte` predicate.

Camera Path

`rvo-camera` opens a `VideoCapture` from either a device index (integer) or a URI string (RTSP, file path, HTTP stream).

For every successful read: 1. Create `Frame { ts: Instant, id: u64, image: Mat }`. 2. `try_send` through a bounded channel (capacity 5). 3. On full channel: increment `METRICS.frame_drops` and discard.

Camera failures are logged with a consecutive-failure counter to avoid log spam. The thread does not panic on open or read failure.

Frame Buffer

`rvo-buffer` holds a fixed-capacity circular buffer of `Frame`.

- `push(frame)` overwrites the oldest slot (O(1), no allocation).
- `slice(start, end)` returns timestamp-sorted frames in the window.
- `newest_frame()` / `newest_instant()` return `Option` (safe on empty buffer).

The buffer is wrapped in `Arc<Mutex<FrameBuffer>>` and shared between the Scheduler (writes, tick-driven) and the ClipManager worker thread (post-roll reads). The lock is held only for the brief push/read operations, so contention between the scheduler tick and the worker is minimal.

Detector Model

```
pub trait DetectorNode: Send {
    fn meta(&self) → DetectorMeta;
    fn execute(&mut self, ctx: &DetectorContext<'_>) → DetectorResult;
    // id(), max_fps(), dependencies(), output_signals(), cost_hint(),
    // requires_frame() all delegate to meta()
}
```

`DetectorContext` carries `now_ns: u64` and `frame: Option<&Frame>`.

`DetectorMeta` declares: `id`, `max_fps`, `dependencies: &'static [SignalType]`, `output_signals: &'static [SignalType]`, `cost_hint`, `requires_frame`.

Synthetic detectors: - `DummyDetector`: 30 FPS, no frame, emits `SignalType::Dummy = 1` with 1s TTL. -

`LoadDetector`: 10 FPS, no frame, busy-spins for `busy_ns`, emits nothing. - `JitterDetector`: 30 FPS, no frame, busy-spins for random ≤ 2 ms.

Scheduler

`rvo-scheduler` is the central orchestration loop.

Each `tick()`:

1. Drain all pending frames from the camera channel into the frame buffer.

2. Increment `rvo_scheduler_ticks`.
3. Snapshot `latest_frame` (holds the newest frame timestamp for dependency checks).
4. For each detector, evaluate in order:
 - Disabled? (set by `DetectorHealth::Failed`)
 - FPS cap elapsed?
 - Load-shedding backoff active?
 - Frame required but buffer empty?
 - All declared signal dependencies fresh?
5. Execute, measure elapsed nanoseconds, store produced signals.
6. On `Failed` health: disable detector permanently until reload.
7. For each event emitted by the event engine: publish, trigger clip job.

Load Shedding

When a detector's last execution time exceeds $2 \times$ its FPS-budget interval, a backoff period is applied based on cost hint:

- `Low`: no backoff — always allowed to recover.
- `Medium`: 100 ms backoff.
- `High`: 500 ms backoff.

During backoff the detector is skipped. The scheduler does not accumulate a queue of missed executions. This keeps the live path from falling behind when a model runs slow.

Signal Store

`rvo-signals` stores one slot per `SignalType` in a fixed `Vec<SignalSlot>`.

Types: `Dummy`, `MotionLevel`, `FacePresent`, `PersonDetected`.

Each slot uses a seqlock-style version counter. `upsert` takes `&mut self` so writes are already serialised by the borrow checker; the version check is defensive for a future move to concurrent writes.

Freshness rule: `signal.ts_ns + signal.ttl_ns < now_ns` → signal is absent.

Condition DSL

`rvo-events` exposes a composable condition type:

```
pub enum CompareOp { Gte, Gt, Eq, Lt, Lte }

pub struct SignalPredicate {
    pub signal_type: SignalType,
    pub op: CompareOp,
    pub value: u64,
}

pub enum Condition {
    All(Vec<SignalPredicate>), // all predicates must hold (AND)
    Any(Vec<SignalPredicate>), // any predicate must hold (OR)
}
```

`Condition::single_gte(signal_type, value)` is the shorthand for the common single-signal greater-than-or-equal case.

A missing or stale signal evaluates to `false` for any predicate that reads it.

Event Engine

One `EventMachine` per `EventDefinition`. State machine:

```
Idle → (condition true) → Potential { start_ns }
    → (duration elapsed) → emit Event + Cooldown { until_ns }
    → (cooldown elapsed) → Idle
```

`EventDefinition`: `event_type`, `condition: Condition`, `duration_ns`, `cooldown_ns`.

`EventEngine::update()` returns `Vec<Event>`. The scheduler iterates and for each event: increments `rvo_events_emitted_total`, calls `EventPublisher`, calls `ClipManager`.

Evidence Pipeline

`ClipManager` uses two stages:

on_event (called on the scheduler tick thread): 1. Lock buffer, read `newest_instant()` → None: drop, count `rvo_clip_drops_total`. 2. Compute clip window `[event_ts - before, event_ts + after]` and `fire_at = now + after`. 3. `try_send` a `PendingJob` into the bounded pending queue (capacity 16). 4. On full pending queue: drop, count `rvo_clip_drops_total`. Never blocks, never spawns a thread.

Single worker thread (runs for the lifetime of `ClipManager`): 1. Drain `PendingJob`s from the pending queue. 2. Sleep until `fire_at` (post-roll window closes). 3. Lock buffer, `slice(start, end)` frames, release lock. 4. `try_send` (`ClipJob`, `Vec<Frame>`) to encoder. On full encoder queue (capacity 8): count `rvo_clip_drops_total`.

Encoder worker:

1. Create `{clips_dir}/{EventType}-{ts_ns}/` directory.
2. Write each frame as `frame_NNNN.jpg` via `opencv::imgcodecs::imwrite`.
3. Write `meta.json`: event type, timestamp ns, frame count, written count, per-frame timestamp array, clip window, encoding latency ms.

Remote Detectors

`rvo-remote` implements `RemoteGrpcDetector`, which sources signals from an external gRPC model service instead of computing in-process.

The gRPC client runs on a dedicated worker thread with its own Tokio runtime and a persistent HTTP/2 channel.

`execute()` is non-blocking:

1. Write newest frame into a single-slot mailbox (overwrite-newest).
2. Return the most recently cached result (stamped with `ts_ns = now - result_age`).

The worker thread loops: take newest frame → JPEG-encode → call `Detect()` → store result. The scheduler tick never touches the network.

Staleness is handled by the existing `SignalStore` TTL: if the worker falls behind, the cached result expires naturally. Transient RPC errors are tolerated; after a consecutive-failure threshold the detector flips to `DetectorHealth::Failed` and the scheduler disables it permanently (until reload).

Config: add a `remote_grpc` entry in `config/rvo.yaml`, or pass `--detector ENDPOINT=SIGNAL` on the CLI. The proto contract is `crates/rvo-remote/proto/detector.proto`.

Event Publishers

Channel logger (`start_event_logger`): always active; logs to stdout.

File sink (`start_event_file_sink`): active when `event_log` is set. Appends one JSON line per event to the configured path, flushes after each write. Format:

```
{"event_type":"DummyEvent","ts_ns":1234567890,"confidence":1.0}
```

Metrics

`http://127.0.0.1:9090/metrics` — Prometheus text format. `http://127.0.0.1:9090/health` — 200 ok while process is alive.

Counters:

Metric	Description
<code>rvo_scheduler_ticks</code>	Total scheduler tick calls
<code>rvo_detector_exec_total</code>	Detector executions
<code>rvo_detector_skip_total</code>	Detector skips (FPS cap, backoff, deps)
<code>rvo_detector_failure_total</code>	Detectors disabled by Failed health
<code>rvo_detector_exec_ns_total</code>	Aggregate detector execution nanoseconds
<code>rvo_events_emitted_total</code>	Confirmed events emitted
<code>rvo_frame_drops_total</code>	Frames dropped (camera channel full)
<code>rvo_clip_drops_total</code>	Clip jobs dropped (encoder queue full or no frames)
<code>rvo_event_drops_total</code>	Events dropped (publisher channel full)

Hot Reload

`SIGHUP` (Unix only): reload config from the path used at startup, rebuild detectors and event engine, swap into the running scheduler atomically. Invalid configs keep the current runtime active and log the error.

Build and Run

CI: `.github/workflows/ci.yml` runs `cargo check --workspace`, `cargo test --workspace`, `cargo clippy -- -D warnings`, `cargo fmt --check`.

```
# Run with default config
cargo run -p rvo-bin

# Override config path
RVO_CONFIG=/path/to/config.yaml cargo run -p rvo-bin

# Observe
curl http://127.0.0.1:9090/metrics
curl http://127.0.0.1:9090/health
tail -f events.jsonl           # if event_log is configured
ls clips/                     # clip output directories
```

RVO Developer Guide

This document explains why RVO is built the way it is: the language choice, the system invariant, the crate decomposition, the layered architecture, and the hard design calls with their tradeoffs. It is written for engineers who need a deep understanding of the system, not a tour of the directory tree.

1. Why Rust

Predictable latency is the first requirement

A garbage-collected runtime periodically stops the world to reclaim memory. For a realtime video pipeline where the live path runs on a 1 ms tick, a GC pause is not a performance regression — it is a correctness failure. Rust eliminates this by having no runtime allocator overhead on the hot path and no GC.

This is not a theoretical concern. Any language with a managed heap (Go, Java, Python) introduces latency variance that is bounded by the GC's pause time, not by the application's logic. Rust's ownership model forces memory to be reclaimed at a known point — the end of its owner's scope — which is deterministic.

Ownership makes concurrency correct by construction

RVO shares exactly one piece of mutable state between the live path and the evidence path: the frame buffer, wrapped in `Arc<Mutex<FrameBuffer>>`. Rust's type system enforces that:

- `Arc` is the only way to share ownership across threads.

- `Mutex` is the only way to get a mutable reference to the inner value.
- The borrow checker prevents holding a `MutexGuard` across an `await` or across a blocking call — enforcing the lock discipline that makes the design correct.

In Go or C++, this discipline is a comment. In Rust, it is a compile error.

The `Send` and `Sync` marker traits mean the compiler will reject any type that is not safe to move across thread boundaries. `DetectorNode: Send` is not a convention — it is enforced on every detector implementation.

Zero-cost abstractions over detectors

`DetectorNode` is a trait object (`Box<dyn DetectorNode>`). Dynamic dispatch has a pointer-indirect call overhead, but no heap allocation per call and no boxing on every invocation. The trait object itself is allocated once at startup. On the hot scheduler path, the cost is one vtable lookup per detector per tick, which is acceptable.

Iterator chains, `map`, `filter_map`, and `try_send` all compile down to the same machine code as a hand-written loop. There is no “cost” to expressing the scheduler logic at the level of the problem.

Cargo workspace maps to the component model

Each crate in the workspace is a unit of compilation, linkage, and dependency. Adding `rvo-testkit` as a `dev-dependency` of `rvo-scheduler` means test infrastructure never enters the production binary. This is not achievable with a flat module layout in a single crate.

crossbeam channels over std channels

`crossbeam_channel::bounded` is the backbone of every bounded queue in the system. Compared to `std::sync::mpsc`:

- Both `Sender` and `Receiver` are `Clone` — multiple producers are trivial.
- `try_send` is lock-free on the fast path.
- The bounded channel semantics match what the system needs: reject and count, never block.

2. What RVO Is

The problem

Live video AI pipelines have a structural tension. The live path — camera read → frame decode → detector inference → signal publish — must run at a fixed FPS. The evidence path — clip capture → frame encode → file write — is slow and variable. Naive implementations couple these, causing the live path to stall behind a slow encoder or a disk flush.

A second problem: model inference is non-deterministic in latency. A detector that usually runs in 5 ms may occasionally take 50 ms. Without explicit scheduling policy, a slow run cascades into accumulated backlog.

A third problem: single-frame detections are noisy. A human face appearing for one frame is not an event worth capturing. A face present continuously for 3 seconds is.

The system invariant

The live path must never wait for slow work.

Everything that could be slow — encoding, file I/O, downstream delivery, post-roll capture — is off the live path. The live path is: camera ingestion → frame buffer push → detector execution → signal publication → event evaluation.

Core contract

1. Capture frames without blocking; drop when the channel is full.
2. Maintain a bounded rolling window of frames for evidence extraction.
3. Schedule detector nodes under FPS and latency constraints; shed load rather than accumulate backlog.
4. Publish typed signals with TTL freshness semantics.
5. Evaluate temporal conditions over signals; confirm events over time, not on instantaneous matches.
6. Extract evidence asynchronously; the live path never waits for the clip encoder.
7. Expose metrics for every drop, skip, and failure.

What RVO is not

- A computer vision model or training framework. It runs models; it does not define them.
- A video codec or muxer. It writes JPEG frames; video muxing is future work.
- A general-purpose stream processor like Kafka or Flink. It is a single-process, bounded runtime.
- A durable event log. Events are written to a JSON-lines file; there is no replay or retention guarantee.

3. Crate Layout

`rvo-bin`

Entrypoint and wiring. Reads config, starts the metrics server, instantiates the frame buffer, builds detectors and the event engine, starts the camera thread and encoder worker, optionally starts the event file sink, installs the SIGHUP reload handler, then enters the 1 ms tick loop. Nothing in `rvo-bin` is a library — it is all startup sequencing and integration glue.

`rvo-config`

YAML config loading and validation. The only crate that touches `serde` deserialization. All other crates accept already-validated config structs.

`rvo-core`

Shared time primitives and the `Frame` type. Kept minimal so every other crate can depend on it without pulling in heavier dependencies.

`rvo-camera`

Opens a `VideoCapture` from a device index or URI. Produces `Frame` values and `try_send`s them into a bounded channel. On a full channel, it increments the frame drop counter and discards. The camera thread does not panic on read failure; it logs a consecutive-failure count and retries. This crate has no knowledge of the downstream scheduler.

`rvo-buffer`

`FrameBuffer`: a fixed-capacity circular buffer of `Frame`. `push` is O(1) and overwrites the oldest slot. `slice(start, end)` returns timestamp-ordered frames in the window. Wrapped in `Arc<Mutex<FrameBuffer>>` and shared between the scheduler (writer) and the clip manager (reader). The lock is never held across a blocking operation.

`rvo-detector`

Defines the `DetectorNode` trait, `DetectorMeta`, `DetectorContext`, and `DetectorResult`. Also contains the synthetic detectors used in production config (`DummyDetector`, `LoadDetector`, `JitterDetector`). This crate is the abstraction boundary between the scheduler and any model implementation.

`rvo-remote`

Bridges RVO to external model services. `RemoteGrpcDetector` implements `DetectorNode` but sources its signals from a gRPC service (the `rvo.detect.v1.Detector` contract in `proto/detector.proto`) instead of computing them in-process. The gRPC client runs on a dedicated worker thread with its own Tokio runtime and a persistent HTTP/2 channel; `execute()` itself never blocks (see §5.9). This is how a camera frame reaches an external YOLO / image-pipeline service and comes back as a `PersonDetected` or `FacePresent` signal. Codegen uses `tonic-build` with a vendored `protoc`, so no system protobuf compiler is required.

`rvo-signals`

Defines `SignalType`, `Signal`, and `SignalStore`. The store is a typed blackboard: one slot per `SignalType`, O(1) read and write by type index, TTL freshness enforced at read time. Signals are the shared language between detectors and the event engine.

`rvo-events`

Implements the condition DSL (`All` / `Any` over `SignalPredicate`), `EventDefinition`, and the temporal state machine per event (`EventMachine`). Also contains `EventPublisher` and the JSON-lines file sink. The event engine consumes the signal store, not raw frames.

`rvo-scheduler`

The orchestration loop. Each `tick()` drains camera frames into the buffer, snapshots the newest frame, evaluates each detector (FPS gate, backoff gate, dependency gate, frame-required gate), runs eligible detectors, stores produced signals, runs the event engine, and dispatches events to publishers and the clip manager. This is the core of the runtime.

rvo-clips

`ClipManager`: receives clip jobs triggered by the event engine. `on_event` is non-blocking: it anchors the clip window to the newest frame timestamp and enqueues a `PendingJob` into a bounded queue (capacity 16). A single long-lived worker thread drains this queue — for each job it sleeps until the post-roll window closes, then locks the frame buffer briefly to slice frames, and hands them to the encoder via `try_send`. No per-event thread is spawned. The encoder worker writes JPEG frames and a `meta.json` sidecar. All of this is off the live path.

rvo-metrics

Global atomic counters and an HTTP server. `/metrics` serves Prometheus text format. `/health` returns `200 ok` as a liveness check. Counters cover frame drops, scheduler ticks, detector executions, skips, failures, aggregate latency, event emissions, clip drops, and event drops.

rvo-testkit

Synthetic test infrastructure: `SyntheticCamera`, `ScriptedDetector`, `ProbabilisticDetector`, `LatencyDetector`, `FailingDetector`, `ChainedDetector`, `EventCapture`, `MetricsSnapshot`, `PipelineBuilder`. This crate is a `dev-dependency` only and never enters a production binary.

rvo-scenarios

End-to-end integration tests using `rvo-testkit`. Each scenario builds a complete pipeline and asserts temporal behavior, drop counts, or detector health transitions.

4. End-to-End Data Flow

```

Camera / RTSP Stream
|
v [bounded channel, cap 5 - drop on full → rvo_frame_drops_total]
Scheduler.tick() [1 ms loop]
|
+--> FrameBuffer [Arc<Mutex<>>, circular, cap 300 - overwrite oldest]
|
+--> DetectorNode(s) [sequential, FPS-gated, backoff-gated, dep-gated]
|   |
|   v
|   SignalStore [typed slots, TTL freshness, 0(1) read/write]
|
+--> EventEngine [Condition DSL: All/Any over SignalPredicates]
|   |
|   v [EventMachine: Idle → Potential → emit → Cooldown → Idle]
|   Vec<Event>
|
+--> EventPublisher [try_send, cap 64 - drop on full → rvo_event_drops_total]
|   |
|   +--> stdout logger
|   +--> JSON-lines file sink (if configured)
|
+--> ClipManager [try_send to pending queue, cap 16 - drop on full → rvo_clip_drops_total]
|   |
|   v [single worker thread: sleep until fire_at, lock buffer briefly, slice frames]
|   | [try_send to encoder, cap 8 - drop on full → rvo_clip_drops_total]
|   Encoder Worker
|   |
|   v
|   clips/{type}_{ts}/frame_NNNN.jpg + meta.json

```

Bounded queue table

Handoff	Capacity	Overflow behavior
Camera → Scheduler	5 frames	Drop, count <code>rvo_frame_drops_total</code>
Scheduler → EventPublisher	64 events	Drop, count <code>rvo_event_drops_total</code>
Scheduler → ClipManager (pending queue)	16 jobs	Drop, count <code>rvo_clip_drops_total</code>
ClipManager worker → Encoder	8 jobs	Drop, count <code>rvo_clip_drops_total</code>
FrameBuffer	300 frames (~10 s at 30 fps)	Overwrite oldest

No unbounded queue exists anywhere.

5. Key Design Decisions and Tradeoffs

5.1 Bounded channels and explicit frame drops

Every channel is bounded and uses `try_send`. When a channel is full, the sender discards and increments a metric counter.

The alternative — blocking the producer until the consumer catches up — turns downstream slowness into upstream stalls, which propagates back to the camera thread and breaks the realtime contract. Dropping is the correct choice when freshness matters more than completeness. A 1-second-old frame is worth less than a fresh one; it should be discarded, not queued.

Frame drops are a signal of overload, not a bug. The metrics surface them explicitly so operators can observe and respond.

The capacity choices are deliberate: - Camera channel at 5: enough to absorb a single slow tick without dropping, not enough to buffer stale frames. - Event publisher at 64: events are small, and bursts are short. A sink stall should not drop events immediately. - Clip queue at 8: encoding is slow. A small queue prevents runaway memory growth from queued raw frames.

5.2 Frame buffer lock discipline

The `Arc<Mutex<FrameBuffer>>` is the only shared mutable state between the live path and the evidence path. The invariant is: the lock is held only for the duration of a buffer operation, never across a blocking call.

The scheduler holds the lock briefly to push a frame and to read the newest timestamp. When an event fires, `on_event` holds the lock briefly to read `newest_instant()` (anchoring the clip window), then releases immediately — no sleeping is done under the lock. The single post-roll worker holds the lock briefly to call `slice()`, and only after sleeping until the post-roll window closes. The lock is never held while sleeping.

This means there is at most one thread (the worker) competing with the scheduler for the frame buffer lock, and that contention window is a single `slice()` call — not the entire post-roll duration. The previous design (one thread per event) would have had N threads competing simultaneously at the moment their post-roll windows expired; that unbounded concurrency no longer exists.

5.3 Sequential detector execution

All detectors run sequentially within a single `tick()` call. This is a deliberate simplification: no thread pool, no work stealing, no concurrent signal writes.

The tradeoff: a single slow detector delays all subsequent detectors in the tick. The mitigation is load shedding — slow detectors are backed off, so they do not run every tick. For a pipeline where each detector is expected to run in milliseconds, sequential execution is simpler and has lower overhead than the coordination cost of a thread pool.

Parallel execution would require concurrent signal writes, which would require the signal store to use atomics or fine-grained locking rather than the current `&mut self` borrow model. That complexity is reserved for a future multi-stream or distributed phase.

5.4 Signal store as typed blackboard with TTL freshness

The signal store is not a queue. It answers one question: *What is the latest valid value of signal X right now?* There is one slot per `SignalType`, and a new write overwrites the previous value. Reads are O(1) by type index.

Freshness is enforced via TTL at read time: `signal.ts_ns + signal.ttl_ns < now_ns` means absent. This prevents stale signals from triggering events long after their detector stopped running. A signal that is not refreshed within its TTL becomes invisible to the event engine.

The store uses a seqlock-style version counter on each slot. Writes increment the version before and after the value update. Reads check that the version is even (no write in progress) and consistent across the read. Currently, writes are serialized by the `&mut self` borrow on `SignalStore`, so the seqlock is defensive against a future move to concurrent writes. If that changes, the memory ordering on the version counter must be reviewed carefully — seqlocks require at minimum `Release` on write and `Acquire` on read.

5.5 Temporal event confirmation

Events are confirmed over time, not on instantaneous signal matches. The state machine per event definition:

```
Idle
→ (condition first evaluates true) → Potential { start_ns }
→ (condition still true, elapsed ≥ duration_ns) → emit Event + Cooldown { until_ns }
→ (cooldown elapsed) → Idle

(condition becomes false while Potential) → Idle
```

This eliminates the noise problem: a one-frame detection never fires an event. The event fires only if the condition holds continuously for `duration_ns`. The cooldown prevents re-triggering on the same sustained condition.

The tradeoff: added complexity in the event engine, and tests that validate event timing must run with real elapsed time (no clock injection). The correctness benefit — eliminating noisy one-shot events — is the core value proposition of the temporal confirmation layer.

5.6 Load shedding via cost hints and overrun detection

When a detector's last execution time exceeds 2× its FPS-budget interval, the scheduler applies a backoff:

- `CostHint::Low`: no backoff — always allowed to recover immediately.
- `CostHint::Medium`: 100 ms backoff.
- `CostHint::High`: 500 ms backoff.

During backoff, the detector is skipped entirely. The scheduler does not queue missed executions. This keeps the live path from falling behind when a model runs slow: RVO degrades by doing less work per unit time, not by accumulating a backlog of stale work.

The cost hint is self-declared by the detector, not measured. This is intentional: the scheduler needs the hint before execution to make admission decisions. The overrun check corrects for detectors that underestimate their cost.

A known limitation: the current load shedding policy only uses execution latency as input. It does not account for downstream queue depth (clip queue depth, event queue depth). A detector that produces many events per tick could saturate the event publisher even under acceptable execution latency.

5.7 Evidence pipeline as best-effort

The clip pipeline is explicitly best-effort. `ClipManager::on_event` uses `try_send` and drops on failure. The encoder worker thread runs at its own pace and does not signal back to the scheduler. A failed or slow encoding does not affect the scheduler tick rate.

The philosophical point: an event is meaningful even if the evidence capture fails. The event itself is the ground truth; the clip is supporting material. Coupling event reliability to encoding reliability would be the wrong abstraction boundary.

5.8 Hot reload and event machine state

SIGHUP (Unix only) reloads config, rebuilds detectors and the event engine, and swaps them into the running scheduler. Invalid configs preserve the live runtime.

A consequence: the event state machines are rebuilt from scratch on reload. Any in-progress `Potential` state is lost. Events that were building toward confirmation are reset to `Idle`. This is acceptable for config changes but should be documented as a known behavior.

5.9 Remote detectors over gRPC (decoupled inference)

A detector does not have to compute in-process. `RemoteGrpcDetector` (in `rvo-remote`) wraps an external model service: each camera frame is JPEG-encoded and sent over gRPC, and the reply is mapped back into a `Signal`. This is how RVO fans frames out to a YOLO service, a face/emotion service, or any process speaking the `rvo.detect.v1.Detector` contract — without coupling the orchestrator to a model runtime or language.

The hard constraint is the system invariant from §2: *the live path must never wait for slow work*. A network round-trip plus model inference is the slowest thing in the system, and the scheduler times every `execute()` call and backs off detectors that overrun (§5.6). So a naive “call gRPC inside `execute()` and block” would either stall the 1 ms tick or trip its own backoff every frame. The design decouples inference from the tick:

scheduler tick (hot)	worker thread (own Tokio runtime + persistent channel)
<code>execute(ctx):</code>	<code>loop:</code>
publish newest frame →	take newest frame, JPEG-encode
read cached result ←	Detect() over the persistent HTTP/2 channel
return signals (w/ ttl)	store (value, produced_at)

- **`execute()` is non-blocking.** It writes the newest frame into a single-slot mailbox (overwrite-newest, the same discipline as the camera→buffer handoff) and returns the most recently cached result. The network never touches the hot path.
- **Staleness is handled by the existing TTL mechanism, not a new one.** The cached result is stamped `ts_ns = now_ns - result_age`, so if the worker falls behind, the signal expires in the `SignalStore` (§5.4) exactly as an in-process signal would. Events never fire on a stale remote detection.
- **The channel is persistent and dialed lazily.** `connect_lazy` means construction never blocks; tonic reconnects transparently, so a service that is down at startup or bounces mid-run does not tear down the detector.
- **Failure is bounded, then terminal.** Transient RPC errors/timeouts are tolerated; only after a threshold of consecutive failures does the detector report `DetectorHealth::Failed`, at which point the scheduler disables it (§5.6 health gate) and the rest of the pipeline keeps running. This is the “kill a model service, the pipeline survives” property.

Tradeoffs: signals lag real-world state by roughly one round-trip, and the worker introduces one thread + one Tokio runtime per remote detector. Both are acceptable for the value — arbitrary models in any language, hot-swappable behind a stable proto, with the orchestrator’s latency guarantees intact. The mapping is deliberately one detector ↔ one `SignalType`, so adding a second model is just a second `remote_grpc` entry (see §7).

6. Known Caveats and Open Problems

Clock injection is absent

The scheduler uses `Instant::now()` directly, not an injected clock. This means tests that validate temporal behavior — event confirmation, TTL expiry, backoff duration — must run with real elapsed time. A test validating a 500 ms backoff must actually wait 500 ms. This makes temporal tests inherently slower and introduces flakiness on a loaded CI machine.

The fix is a `Clock` trait that the scheduler accepts, with a `MockClock` in testkit. This is a non-trivial refactor because `Instant` is used in many places across the scheduler and event engine.

Sequential detector execution creates a bottleneck

All detectors share the same scheduler tick thread. A detector that takes 30 ms on a given tick delays every subsequent detector in that tick by 30 ms, even if their individual FPS budgets would have allowed them to run. Load shedding mitigates this by backing off the slow detector on the next tick, but it does not address the current tick’s delay.

In a pipeline where high-cost detectors run at low FPS alongside low-cost detectors at high FPS, this can cause the low-cost detectors to miss their timing windows on ticks where a high-cost detector runs.

Bounded pending-queue depth limits burst clip capture

`ClipManager` uses a single worker thread and a bounded pending queue (capacity 16). A burst of more than 16 confirmed events in flight simultaneously will start dropping clip jobs, counted as `rvo_clip_drops_total`. This is

intentional — the old design spawned one thread per event, which was the only unbounded resource path in the system. The current design trades worst-case clip capture completeness for strict resource bounds. If 16 is too small for a high-event workload, `PENDING_CAP` can be increased, but memory growth is proportional to that capacity times the post-roll duration.

Global metric atomics in parallel tests

`rvo-metrics` uses global atomic counters shared across all test threads. In a parallel test run, counter deltas from different tests intermingle. A test that asserts `rvo_frame_drops_total == 2` may see a higher value because another test dropped frames concurrently.

The workaround is `cargo test -- --test-threads=1`. The fix requires per-instance metrics objects rather than global atomics, which is a larger refactor that affects the metric API across all crates.

Post-roll clip timing requires test accommodation

`ClipManager` uses a single worker thread that sleeps until each job's `fire_at` before slicing the buffer. Any test that reads `clip_rx` must wait longer than `clip_after` before asserting, since there is no signal back to the test when the worker finishes sleeping.

Until clock injection is implemented, tests must use a short but non-zero `clip_after` and sleep at least that long before asserting on clip output.

Load shedding ignores queue depth

The backoff policy uses only execution latency as input. A detector that runs fast but produces a burst of events can saturate the event publisher channel (capacity 64) without triggering any backoff. Similarly, a `ClipManager` that spawns many post-roll threads can exhaust the encoder queue (capacity 8) without affecting detector scheduling.

Incorporating downstream queue depth into the shedding policy would make the system more holistically self-limiting under load. For example, a detector that fires many events per tick could saturate the `EventPublisher` channel (capacity 64) or fill the `ClipManager` pending queue (capacity 16) without affecting detector scheduling.

Static lifetime on `DetectorMeta` output lists

`DetectorMeta.output_signals` and `dependencies` are typed as `&'static [SignalType]`. This is correct for detectors whose signal lists are compile-time constants. For test detectors with dynamic output lists, the only options are:

1. Declare the slice as a module-level `static`.
2. Leak a `Box<[SignalType]>` to obtain a `&'static` reference.

Option 2 is acceptable in test-only code but is a memory leak that grows with the number of test detector instances. If the API is ever extended to support dynamically configured detectors, the lifetime constraint must be relaxed, likely by switching to a `Vec<SignalType>` owned by the `DetectorMeta` struct.

7. Running RVO: the CLI and the TUI

RVO ships two front-ends, both in `rvo-bin`: a headless **CLI** (`rvo`, the default binary) for servers and scripted demos, and an interactive **TUI** (`rvo-tui`) for driving and watching the pipeline live. Both run the same runtime — they only differ in how you start it and what you see.

7.1 The CLI app (`rvo`)

HOW TO USE IT

```
# Default: read config/rvo.yaml (or $RVO_CONFIG), run until killed.
cargo run -p rvo-bin

# Discover camera options before starting.
cargo run -p rvo-bin -- --list-cameras
#   device 0 (use: --camera-device 0)

# Pick a camera and attach remote model services from the command line.
cargo run -p rvo-bin -- \
  --camera-device 0 \
  --detector http://localhost:50051=PersonDetected \
  --detector http://localhost:50052=FacePresent \
  --clips-dir clips/demo
```

Flag	Purpose
<code>--config <PATH></code>	Config file path (overrides <code>\$RVO_CONFIG</code>).
<code>--camera-device <N></code>	Use local device index <code>N</code> .
<code>--camera-uri <URI></code>	Use an RTSP/file/MJPEG URI (mutually exclusive with <code>--camera-device</code>).
<code>--detector ENDPOINT=SIGNAL</code>	Add a <code>remote_grpc</code> detector. Repeatable.
<code>--clips-dir <DIR></code>	Output directory for evidence clips.
<code>--metrics-port <PORT></code>	Prometheus/health server port (default 9090).
<code>--list-cameras</code>	Probe device indices 0–9, print which open, exit.

HOW IT WORKS

`main()` parses flags with `clap`, then: resolves the config path (`--config` → `$RVO_CONFIG` → `config/rvo.yaml`), loads and validates the YAML, and applies CLI overrides on top of the parsed `RvoConfig` — camera source, clips dir, and any `--detector` specs appended as `remote_grpc` `DetectorConfig` entries. The merged config then flows through the normal wiring (`build_detectors`, `build_event_engine`) into the scheduler's 1 ms tick loop.

The division of labour is deliberate: **CLI flags augment the camera and add detectors; events always come from the config file**. This keeps the event rules (which need durations, cooldowns, thresholds) declarative while letting you point the binary at a different camera or model service without editing YAML. `--list-cameras` is backed by `rvo_camera::list_cameras`, which best-effort-opens each device index so you know what `--camera-device` values are valid. Note that SIGHUP reload re-reads the file and therefore drops CLI overrides — the file is the source of truth on reload.

7.2 The TUI app (`rvo-tui`)

HOW TO USE IT

```
cargo run -p rvo-bin --bin rvo-tui
# or against the gRPC demo config:
RVO_CONFIG=config/rvo-remote.yaml cargo run -p rvo-bin --bin rvo-tui
```

Two phases:

- **Menu** — pick a camera source. Probed local devices and the config default are listed; `↑/↓` (or `j/k`) to move, `Enter` to start, `q` to quit.
- **Dashboard** — a live view while the pipeline runs: configured **services**, the **signal** blackboard (each `SignalType` shown present/absent with its current value), aggregate **metrics** (ticks, detector exec/skip/fail, average exec latency, events, drops), and the **recent events** stream. `q` / `Esc` quits.

HOW IT WORKS

The detectors and events still come entirely from config; the menu only chooses the camera. On start, the TUI builds the same runtime as the CLI and spawns the scheduler tick loop on a background thread, then the UI thread renders at ~7 fps using `ratatui`. It reads three live sources without disturbing the hot path:

- **Metrics** — the global atomic counters in `rvo-metrics` (`METRICS`), read directly each frame.
- **Signals** — `Scheduler::signal_snapshot()` briefly locks the scheduler to read each slot's non-expired value (the same TTL check the event engine uses).
- **Events** — a tap thread drains the event channel into a 50-entry ring buffer the dashboard renders from, instead of the stdout/file sink the CLI uses.

One detail worth noting: the TUI sets `RVO_REMOTE_SILENT` so a remote detector whose service is down cannot spam stderr over the alternate screen. The same failure still surfaces — as a rising `detector_fail` count and, after the failure threshold, the detector dropping out — just without corrupting the display.

7.3 The end-to-end gRPC demo

`demo/` wires the whole story together with two trivial stub model services (see `demo/README.md`):

```
pip install -r demo/requirements.txt
bash demo/run_demo.sh           # codegen stubs, start both services, run RVO
```

Then cover/uncover the camera to drive the signals and observe `curl :9090/metrics`, `tail -f events.jsonl`, and `ls clips/demo/`. The stub services implement the same `rvo.detect.v1.Detector` proto as a real model would, so the demo exercises the actual remote-detector path (\$5.9), not a mock.

7.4 The web POC (`rvo-web`)

For the most legible “what does RVO do” demo — and to make the pluggable model story tangible to a new user or an interviewer — `rvo-web` serves a browser dashboard:

```
cargo run -p rvo-bin --bin rvo-web           # then open http://127.0.0.1:8080
# RVO_CONFIG=config/rvo-remote.yaml RVO_WEB_PORT=9000 cargo run -p rvo-bin --bin rvo-web
```

The page shows the camera source, the live signal blackboard (chips that light up green when present), aggregate metrics, registered model nodes, and recent events — polling `GET /api/state` every 700 ms. The key interaction is **adding a model node from the browser**: enter a gRPC `endpoint` + the `signal` it produces and submit, which `POST /api/nodes` turns into a `RemoteGrpcDetector` injected into the running scheduler via `Scheduler::add_detector` (\$7 wiring). No restart — the node appears and its signal lights up as soon as the service responds. This is the camera→RVO→models fan-out made visible.

How it works: `rvo-web` builds the same runtime as the CLI, runs the scheduler tick loop on a background thread, and reuses the `tiny_http` server (no new web framework). It reads live state from `Scheduler::signal_snapshot()`, the global `METRICS`, and an event-tap ring buffer; the frontend is a single embedded HTML/JS page with no external assets, so it works offline. Routes: `GET /` (page), `GET /api/state` (JSON), `POST /api/nodes` (add a node), `GET /metrics` (Prometheus, same as the CLI server).

Because a model node requires camera frames to act on, a webcam-less host should point `camera.source_uri` at a video file in the config so frames flow and added nodes have something to score.

8. Practical Dev Workflow

Build and run

```
cargo build -p rvo-bin
RVO_CONFIG=config/rvo.yaml cargo run -p rvo-bin
```

Observe the running system

```
curl http://127.0.0.1:9090/metrics
curl http://127.0.0.1:9090/health
tail -f events.jsonl           # if event_log is set in config
ls clips/                      # evidence output directories
```

Run tests


```
cargo test --workspace
cargo test -p rvo-scenarios # integration scenarios only
cargo test -- --test-threads=1 # if metric assertions are sensitive to parallel runs
```

Add a detector

1. Implement `DetectorNode` in `rvo-detector` (production) or `rvo-testkit` (test-only).
2. Declare `meta()` with `id`, `max_fps`, `output_signals`, `dependencies`, `cost_hint`, and `requires_frame`.
3. Wire it into `rvo-bin` or config.
4. Write a scenario test in `rvo-scenarios` if it affects event behavior.

Add a remote model service

1. Implement the `rvo.detect.v1.Detector` gRPC contract in any language (see `demo/services/model_service.py` for a reference). Return `SignalOut` entries whose `signal_type` matches a known `SignalType` name.
2. Add a `remote_grpc` detector to config (`endpoint`, `output_signal`, optional `max_fps` / `timeout_ms` / `ttl_ms`) or pass `--detector ENDPOINT=SIGNAL` on the CLI.
3. No Rust changes are needed for a new model — the proto is the boundary. To validate the integration deterministically, add a test against an in-process tonic mock server (see `crates/rvo-remote/tests/grpc_pipeline.rs`).

Add a signal type

1. Extend `SignalType` in `rvo-signals` (add to `ALL` and `name()` too).
2. Update condition builders or event definitions that reference the new type.
3. Verify freshness and TTL semantics in a scenario test.

Add an event rule

1. Define `EventDefinition` with `condition`, `duration_ns`, and `cooldown_ns`.
2. Register it in `EventEngine::new_many(...)`.
3. Write a scenario that validates both the confirmation duration and the cooldown.

9. Repo Structure

Proto file sync

The `rvo.detect.v1.Detector` proto is defined in two locations:

- **Canonical:** `crates/rvo-remote/proto/detector.proto` — used by the Rust `tonic-build` codegen.
- **Demo copy:** `demo/proto/detector.proto` — used by the Python stub services in `demo/services/`.

The demo copy carries a comment (`// Keep in sync with crates/rvo-remote/proto/detector.proto`) but sync is currently manual. When changing the proto, update both files. The canonical copy has full doc comments; the demo copy intentionally has minimal comments. A CI check enforcing field-level equivalence is future work.

Cargo.toml	workspace members
config/	
rvo.yaml	default runtime config
rvo-remote.yaml	demo config: two remote_grpc detectors
crates/	
rvo-bin/	entrypoint
src/main.rs	rvo CLI (default binary)
src/bin/rvo-tui.rs	rvo-tui interactive dashboard
src/bin/rvo-web.rs	rvo-web browser dashboard + node-add API
examples/	synthetic_demo, rtsp_demo
rvo-config/	YAML loading
rvo-core/	Frame, time primitives
rvo-camera/	capture (+ list_cameras probe)
rvo-buffer/	circular frame buffer
rvo-detector/	trait and synthetic detectors
rvo-remote/	gRPC remote detector (proto + tonic client)
rvo-signals/	typed signal store
rvo-events/	condition DSL, event engine, publishers
rvo-scheduler/	orchestration loop
rvo-clips/	evidence pipeline
rvo-metrics/	Prometheus counters, HTTP
rvo-testkit/	test-only infrastructure
rvo-scenarios/	integration tests
demo/	end-to-end gRPC demo
proto/detector.proto	shared service contract
services/model_service.py	trivial stub model service
run_demo.sh, gen_protos.sh	harness
docs/	
ARCHITECTURE.md	data flow, component contracts, design boundaries
BENCHMARKING.md	benchmark suite reference (how to run, scenarios, CSV schema)
BENCHMARK_PERFORMANCE.md	methodology rationale, measured results, architectural claims
CURRENT_IMPLEMENTATION.md	what the running code does today
DEVELOPER_GUIDE.md	this file
LOW_LATENCY_SYSTEMS.md	domain primer: latency, queuing, scheduling, observability
PLOT_GUIDE.md	figure descriptions, axes, and plotting recipes
README.md	project overview and quick start

Low-Latency Systems — Concepts, Terms, and How RVO Applies Them

A self-contained conceptual guide to the domain RVO lives in: realtime, low-latency systems. It defines the vocabulary, explains *why* each idea exists and what role it plays, and grounds every term in a concrete RVO example so the abstraction is never free-floating.

Read it top to bottom for a mental model, or jump to the [Glossary](#) for a quick lookup. RVO-specific framing of these same ideas, with file references, lives in `DEVELOPER_GUIDE.md` (especially §5 design decisions and §6 caveats).

0. The one idea everything hangs off

In a low-latency system, the scarce resource is not CPU or memory — it is time you can predict.

A request that usually takes 2 ms but occasionally takes 200 ms is often worse than one that always takes 10 ms, because everything downstream must budget for the worst case. Low-latency engineering is therefore mostly about **removing variance**, not lowering the average. Almost every technique below — no GC, bounded queues, dropping instead of blocking, off-loop slow work — exists to make the *distribution* of latency narrow and its *tail* bounded.

RVO's invariant is the concrete form of this: **the live path must never wait for slow work.**

1. Latency vs Throughput

- **Latency** — how long one operation takes, end to end. Measured in time (μ s, ms).

- **Throughput** — how many operations complete per unit time (ops/s, fps, QPS).

They are different axes and often in tension. Batching raises throughput but adds latency (you wait to fill the batch). Pipelining can raise both. A system can have high throughput and terrible tail latency, or low latency and low throughput.

Role: decide which you are optimizing *before* you design. RVO optimizes **latency and freshness**; throughput is capped deliberately (FPS gates) so the loop stays responsive.

RVO: control loop ~972 Hz (throughput of the scheduler), but the design target is the per-tick latency staying ~1 ms so detectors run on schedule.

2. Tail latency (the heart of the field)

Averages lie. A mean of 5 ms can hide that 1% of requests take 500 ms. You quote **percentiles**:

- **p50 (median)** — typical experience.
- **p95 / p99** — the slow minority; what most users hit *sometimes*.
- **p99.9 / p99.99** — the “tail.” At scale, every user hits the tail regularly.

Why the tail dominates at scale: if a request fans out to N backends and waits for all of them, its latency is the **max** of N samples. With $N = 100$ and each backend's p99 = 10 ms, the probability that *at least one* is slow is ~63% — so the aggregate's *median* is near the components' p99. This is **tail-at-scale amplification** (Dean & Barroso, “The Tail at Scale”).

Coordinated omission — a subtle measurement bug: if your load generator stops sending while the system is stalled, it never records the latency of the requests that *would* have arrived during the stall, so the measured p99 is far rosier than reality. Always measure latency from the request's *intended* start time, not when the system got around to it.

Role: percentiles + tail-aware design are the language of the field. “We improved p99.9 from 40 ms to 8 ms” is a real claim; “we improved the average” usually isn't.

*RVO: the production `/metrics` endpoint reports aggregate counters (mean exec latency via `rvo_detector_exec_ns_total` / `rvo_detector_exec_total`). The load harness (`rvo-bench`) uses **HdrHistogram** to capture p50/p99/p99.9 of scheduler tick and detector exec latency across 13 benchmark scenarios — this is where the percentile story lives. The gRPC fan-out to multiple model nodes is exactly a tail-at-scale situation — which is why each remote call is decoupled and TTL-bounded (§9) rather than awaited inline.*

3. The fast-path / slow-path split

The single most important architectural pattern for low latency.

- **Fast path (hot path)** — the code that must be quick and predictable, run on every operation. Keep it free of allocation, locks held across I/O, blocking calls, and anything variable.
- **Slow path (cold path)** — anything slow, bursty, or best-effort: disk writes, encoding, network calls, downstream delivery. Push it *off* the hot path, behind a queue, onto another thread.

Role: isolate variance. The slow path can stall, retry, or fall behind without ever touching the hot path's latency distribution.

*RVO: hot path = camera ingest → frame buffer → detector dispatch → signal write → event check. Cold path = clip encoding, file I/O, and — crucially — **remote model inference**, which runs on a worker thread so a slow model never blocks the ~1 ms tick.*

4. Queueing theory you actually need

You don't need heavy math, but two results explain most capacity decisions.

Little's Law

$L = \lambda \times W$ — average number of items in a system = arrival rate $\times$ average time each spends in it.

Rearranged, $W = L / \lambda$. Use it to size queues and reason about latency: if 1000 req/s arrive and each spends 50 ms in the system, there are ~50 in flight on average. If your queue is smaller than that, you'll drop; if much larger, you're hiding latency in a backlog.

Utilization and the hockey stick

For a simple queue (M/M/1), mean response time $\approx S / (1 - \rho)$, where S is service time and ρ is **utilization** (fraction of capacity used). The curve is a hockey stick:

- $\rho = 0.5 \rightarrow$ latency $\approx 2\times$ service time.
- $\rho = 0.9 \rightarrow$ latency $\approx 10\times$ service time.
- $\rho = 0.99 \rightarrow$ latency $\approx 100\times$ service time.

Role: this is *why* low-latency systems run at moderate utilization (often < 70%) and shed load before saturation. Chasing 100% utilization trades a little efficiency for a catastrophic latency tail.

RVO: the tick budget is ~1 ms. A detector needing 3 ms can't fit, so the scheduler FPS-caps it and backs it off on overrun — keeping effective utilization of the loop low enough that detectors run on time. Capacity $\approx \Sigma(\text{detector_fps} \times \text{service_time})$ must stay under one core's budget.

5. Backpressure

Backpressure is how a system tells (or forces) an upstream producer to slow down when a downstream consumer can't keep up. Without it, a fast producer + slow consumer = an unbounded queue = growing latency and eventual OOM.

Three responses when a buffer fills:

1. **Block** — make the producer wait. Propagates slowness upstream; correct when you must not lose data, dangerous when the producer is realtime (a camera can't "wait").
2. **Drop (shed)** — discard and count. Correct when **freshness > completeness**.
3. **Buffer** — store for later. Only safe if *bounded*; an unbounded buffer just defers the failure and makes it worse.

Role: decide, per queue, which of the three is correct — and make every queue bounded.

*RVO: every handoff is a **bounded** `crossbeam` channel with `try_send` and **drop-and-count** (camera→scheduler cap 5, →events cap 64, →clips cap 8). A full queue increments a `*_drops_total` metric. "Drop-don't-block: a 1-second-old frame is worth less than a fresh one."*

6. Load shedding & admission control

- **Admission control** — deciding *whether* to do a unit of work before starting it.
- **Load shedding** — deliberately *not* doing some work when overloaded, to protect the rest.
- **Graceful degradation** — under overload, the system does *less*, not *worse*; quality drops smoothly instead of the whole thing collapsing.

Role: an overloaded system that keeps accepting everything melts down (latency hockey stick, then OOM). Shedding keeps the surviving work fast.

*RVO: detectors declare a **cost hint** (Low/Medium/High). When one overruns its FPS-budget interval ($2\times$ threshold), the scheduler applies **backoff** (Medium 100 ms / High 500 ms), skipping it rather than queuing missed runs. The observable is `detector_skips`. The identity `skips + execs = ticks × detectors` shows admission control is exhaustive — every slot is explicitly run or shed, never silently lost.*

7. Freshness, staleness, and TTL

In realtime systems, **data has an expiry**. A detection that a person is present is only meaningful for a short window; acting on a 5-second-old detection is a correctness bug, not just a performance one.

- **TTL (time-to-live)** — how long a value stays valid after it's produced.
- **Staleness** — how old the data you're acting on is.

Role: bound staleness explicitly so the system never acts on outdated state. This also lets you decouple producers and consumers safely — a lagging producer's data simply expires instead of triggering wrong actions.

RVO: every `Signal` carries `ts_ns + ttl_ns`; the `SignalStore` returns "absent" once expired. A remote detector stamps its cached result with `ts_ns = now - result_age`, so if the model worker falls behind, the signal expires naturally and events don't fire on stale inferences.

8. Determinism and the runtime (why language/runtime choice matters)

Latency variance often comes from *below* your code:

- **Garbage collection (GC)** — managed runtimes (Java, Go, Python, JS) periodically reclaim memory, sometimes "stop-the-world." A GC pause is unbounded variance injected at random — fatal for a 1 ms loop. This is the #1 reason low-latency systems use non-GC languages (C, C++, Rust) or heavily tuned/region-based allocators.
- **Allocation on the hot path** — even without GC, `malloc` can take a lock or hit the OS. Low-latency code pre-allocates and reuses (object pools, ring buffers, arenas) so the hot path does **zero allocation**.
- **Jitter sources** — OS scheduler preemption, page faults, interrupts, CPU frequency scaling, NUMA effects, cache misses. Mitigations: CPU pinning, huge pages, `mLock`, isolated cores, busy-polling instead of sleeping.

Role: predictable latency requires a predictable substrate. You can't p99-tune on top of a runtime that randomly pauses.

*RVO: written in **Rust** — no GC, deterministic destruction at end of scope, ownership that enforces lock discipline at compile time. The hot path avoids per-tick allocation; trait objects (`Box<dyn DetectorNode>`) are allocated once at startup, not per call.*

9. Concurrency models

How work is split across threads/cores, and how they coordinate.

- **Shared-memory + locks** — threads share state, guarded by `Mutex` / `RwLock`. Simple but lock contention and *holding a lock across slow work* are classic latency killers.
- **Message passing / shared-nothing** — threads own their state and communicate via channels (Actor model, CSP, Go channels). Avoids shared mutable state and most data races. Scales cleanly.
- **Lock-free / wait-free** — algorithms using atomics + CAS so no thread blocks another (ring buffers, `seqlock`). Lowest latency, highest difficulty.
- **Thread-per-core (share-nothing)** — pin one thread per core, partition data so cores never synchronize (Seastar, ScyllaDB, Redis-ish). Eliminates cross-core contention.
- **Async / event loop** — one thread multiplexes many tasks via non-blocking I/O (Tokio, Node). Powerful, but **blocking the event loop** (a sync call inside async) stalls *everything* — a top async footgun.

Key sub-terms:

- **Lock discipline** — never hold a lock across I/O, sleep, or an `await`. Keep critical sections tiny.
- **Atomics & memory ordering** — `Relaxed`, `Acquire`, `Release`, `AcqRel`, `SeqCst` control how memory operations are observed across threads. Stronger ordering = more correctness guarantees = more cost.
- **seqlock** — a reader/writer pattern: the writer bumps a version counter before (odd = write in progress) and after (even = done) updating; the reader reads version, data, version again, and retries if it changed or was odd. Cheap reads, no reader locks.
- **False sharing** — two unrelated atomics on the same 64-byte cache line cause cores to fight over it; fixed by padding/aligning to a cache line.
- **Head-of-line (HOL) blocking** — one slow item stuck at the front delays everything behind it (in a queue, a TCP connection, or a single-threaded loop).

Role: pick the model that matches your latency target and complexity budget.

*RVO: mostly **message passing** (bounded channels between threads) + a single `Arc<Mutex<FrameBuffer>>` with strict lock discipline (lock held only for a push/slice, never across the post-roll sleep). The `SignalStore` uses a **seqlock** version counter. The remote detector runs gRPC on its own thread so the tick loop never blocks. A known*

*limitation: detectors run **sequentially** within a tick, so a slow one is HOL blocking within that tick (mitigated next tick by backoff).*

10. Memory discipline

- **Bounded memory** — usage is a function of a *window* or fixed capacity, not of uptime or input volume. `O(window)`, not `O(uptime)`.
- **Ring / circular buffer** — fixed-size array that overwrites the oldest entry; $O(1)$ push, no allocation, natural for “keep the last N.”
- **Cache locality** — sequential, compact data is faster than pointer-chasing; a cache miss can cost $\sim 100\times$ an L1 hit.

Role: unbounded memory growth is a latency *and* availability bug (GC pressure, swap, OOM). Bound it structurally.

RVO: the `FrameBuffer` is a 300-slot ring (~ 10 s @ 30 fps), overwrite-oldest. Memory is a function of the window, never of how long the process has run.

11. The control-loop pattern

Many realtime systems are a **loop running at a fixed cadence** (a “tick”), rather than purely event-driven. Each tick does a bounded amount of work and the loop targets a fixed frequency.

- **Soft real-time** — missing a deadline degrades quality but isn’t catastrophic (video, trading dashboards).
- **Hard real-time** — missing a deadline is a failure (pacemakers, avionics); requires an RTOS and provable worst-case execution time.

Role: a fixed-cadence loop makes timing predictable and admission control natural (you know how much budget each tick has).

*RVO: a ~ 1 kHz `tick()` loop. It’s **soft** real-time — a late tick drops freshness, not safety. Hard-real-time guarantees would need an RTOS and WCET analysis, which RVO does not claim.*

12. Reliability under load & failure isolation

Low latency is worthless if one slow dependency takes down everything.

- **Bulkhead** — isolate resources (thread pools, queues) per dependency so one failure can’t exhaust the whole system (named after ship compartments).
- **Circuit breaker** — after N failures, stop calling a dead dependency for a while (fail fast) instead of piling up timeouts.
- **Timeout** — never wait unbounded on a dependency; bound every external call.
- **Retry** — re-attempt failed calls — but with **exponential backoff + jitter**, or you cause a **retry storm / thundering herd** that finishes off a struggling service.
- **Failure isolation / graceful degradation** — a failed component is removed; the system continues with reduced function.

Role: convert dependency failure into bounded, local degradation instead of global collapse.

*RVO: each remote detector has a **per-RPC timeout**, tolerates transient failures, and flips to `Failed` only after a threshold — at which point the scheduler **disables** it and the rest of the pipeline keeps running (`detector_failures` counts it). That’s a bulkhead + circuit-breaker-ish pattern: “kill a model service, the pipeline survives.”*

13. Observability

You cannot tune what you cannot measure.

- **SLI (Indicator)** — a measured quantity: latency p99, error rate, drop rate.
- **SLO (Objective)** — the target for an SLI: “p99 < 10 ms, 99.9% of the time.”

- **SLA (Agreement)** — a contractual SLO with consequences.
- **Error budget** — $1 - \text{SLO}$. If your SLO is 99.9%, you have 0.1% to “spend” on failures before you must stop shipping risk.

Metric types:

- **Counter** — monotonic total (requests, drops). You rate it: $\text{rate}(x)$.
- **Gauge** — a current value (queue depth, in-flight count, temperature).
- **Histogram** — a distribution; **required** for latency, because you cannot average percentiles. (p99 of two machines $\neq$ average of their p99s.)

Two standard checklists:

- **RED** (request-driven services): **R**ate, **E**rrors, **D**uration.
- **USE** (resources): **U**tilization, **S**aturation, **E**rrors.

Role: define SLIs/SLOs, expose counters + histograms, and watch saturation (queue depth, drops) as your early-warning signal.

*RVO: exposes Prometheus **counters** (`scheduler_ticks`, `detector_execs`, `detector_skips`, `detector_failures`, `events_emitted`, `*_drops_total`) on `/metrics`. SLI mapping: loop frequency ($\geq$ target Hz), drop rate ($< X\%$), detector failure budget. The honest gap: latency is currently a mean, not a histogram (see §2).*

14. Capacity planning & scaling

- **Vertical scaling** — bigger machine (more cores/RAM). Simple, finite, diminishing.
- **Horizontal scaling** — more machines/instances. Needs the work to partition.
- **Shared-nothing** — instances share no mutable state, so they scale linearly with no coordination cost. The gold standard for horizontal scale.
- **Back-of-envelope capacity** — estimate before you load-test: $\text{capacity} \approx \text{resource} / \text{per-unit cost}$. State assumptions; don't invent a QPS you didn't measure.
- **Fan-out bandwidth** — for a service that pushes data out: $\text{rate} \times \text{payload_size}$.

Role: know your ceiling and your scaling axis before you're asked.

*RVO: **shared-nothing per camera** — one process per stream, no cross-instance state, so it scales linearly across cameras. Model services scale independently behind their gRPC endpoint (replicas + load balancer). Adding a model node is **O(1) on the hot path** (a worker + a channel), so it doesn't degrade loop latency. Per-node fan-out $\approx \text{max_fps} \times \text{frame_size}$ (e.g. 15 fps $\times$ ~30 KB $\approx$ 450 KB/s).*

15. Networking for low latency

Relevant because RVO fans frames out over gRPC.

- **Connection setup cost** — TCP handshake + TLS adds round-trips. **Reuse persistent connections** (connection pooling) instead of dialing per request.
- **HTTP/1.1** — one in-flight request per connection (or fragile pipelining) → HOL blocking; clients open many connections.
- **HTTP/2** — **multiplexes** many streams over one connection (what gRPC uses). Removes application-level HOL, but a single TCP connection still has **TCP-level HOL** (a lost packet stalls all streams). **HTTP/3 / QUIC** (over UDP) fixes that.
- **gRPC + Protobuf vs REST + JSON** — binary framing + a compact schema → smaller payloads, cheaper (de)serialization, streaming, and codegen'd contracts. Lower and more predictable latency than text JSON over HTTP/1. The tradeoff: not human-readable, needs tooling.
- **Serialization cost** — encoding/decoding is real CPU on the hot path; binary formats and zero-copy parsing matter at scale.
- **Sync vs async I/O** — blocking a thread per connection limits concurrency; non-blocking I/O + an event loop scales to many connections, but you must never block that loop.

Role: the wire format and connection model are first-order latency decisions, not afterthoughts.

*RVO: chose **gRPC over HTTP/2** with a **persistent, lazily-dialed channel** per model node (reused across calls, auto-reconnecting). The frame is JPEG-encoded once and sent as bytes. The gRPC client runs on a worker thread (async runtime) so the synchronous tick loop is never blocked on the network.*

16. Common anti-patterns (what *not* to do)

- **Unbounded queues** — defer failure, then OOM. Always bound + decide drop/block.
- **Blocking the hot path / event loop** — one sync call (I/O, lock, sleep) inside the fast path or async executor stalls everything.
- **Holding a lock across slow work** — turns a brief critical section into a global stall.
- **Optimizing the average, ignoring the tail** — the tail is what users and downstreams actually feel.
- **Running at ~100% utilization** — the latency hockey stick; leave headroom, shed early.
- **Retry without backoff/jitter** — turns a blip into a self-inflicted DDoS.
- **Acting on stale data** — no TTL → correctness bugs masquerading as features.
- **Coordinated-omission measurement** — your benchmark says p99 = 3 ms; production says 300 ms.

17. How RVO embodies the domain (one-screen map)

Concept	RVO mechanism
Fast/slow path split	1 kHz tick loop; encoding, I/O, inference off-loop
Bounded queues + backpressure	<code>crossbeam</code> bounded channels, <code>try_send</code> , drop-and-count
Load shedding	cost hints + overrun backoff; <code>detector_skips</code>
Freshness/TTL	<code>Signal.ts_ns + ttl_ns</code> ; expiry at read time
No-GC determinism	Rust; once-at-startup allocation; ownership-enforced locks
Concurrency	message passing + one disciplined <code>Mutex</code> ; seqlock store
Bounded memory	300-slot ring <code>FrameBuffer</code> (~10 s)
Failure isolation	per-RPC timeout, threshold → <code>Failed</code> → detector disabled
Decoupled fan-out	remote detector = worker thread + persistent gRPC channel
Observability	Prometheus counters on <code>/metrics</code> ; HDR histograms (p50/p99/p99.9) in <code>rvo-bench</code> load harness
Shared-nothing scale	one process per camera; O(1) hot-path node add

Glossary

- **Admission control** — deciding whether to start a unit of work before doing it.
- **Backpressure** — mechanism forcing/asking a producer to slow when the consumer lags.
- **Backoff (exponential)** — increasing wait between retries/attempts to avoid storms.
- **Bulkhead** — resource isolation so one failure can't sink the whole system.
- **Circuit breaker** — stop calling a failing dependency for a cooldown; fail fast.
- **Control loop / tick** — fixed-cadence execution of bounded work per cycle.
- **Coordinated omission** — under-measuring latency by not counting requests missed during a stall.
- **Cold/slow path** — non-time-critical work moved off the hot path.
- **Error budget** — `1 - SL0`; the allowable amount of failure.
- **False sharing** — cache-line contention between unrelated data.
- **Fan-out** — one request triggering many downstream calls.

- **Freshness** — how recent the data is; opposite of staleness.
- **Graceful degradation** — doing less under load, not failing entirely.
- **Hard/soft real-time** — deadline miss is catastrophic vs merely quality-reducing.
- **Head-of-line (HOL) blocking** — a stuck front item delaying everything behind it.
- **Histogram** — distribution metric; required to compute latency percentiles.
- **Hot/fast path** — the latency-critical code run on every operation.
- **Jitter** — variance in latency/timing.
- **Little's Law** — $L = \lambda W$; items-in-system = arrival rate $\times$ time-in-system.
- **Load shedding** — intentionally dropping work under overload.
- **Lock discipline** — never hold a lock across slow work; keep critical sections tiny.
- **Lock-free / wait-free** — coordination without blocking, via atomics/CAS.
- **Memory ordering** — `Relaxed/Acquire/Release/AcqRel/SeqCst` visibility guarantees.
- **Percentile (p50/p99/p99.9)** — latency at a rank of the distribution.
- **RED / USE** — Rate-Errors-Duration (services) / Utilization-Saturation-Errors (resources).
- **Ring buffer** — fixed-size, overwrite-oldest circular buffer.
- **seqlock** — versioned read/write pattern for cheap, lock-free reads.
- **Shared-nothing** — instances share no mutable state; scale linearly.
- **SLI / SLO / SLA** — measured indicator / internal target / contractual target.
- **Stale data** — data older than its useful window.
- **Tail latency** — the high percentiles; what dominates at scale.
- **Tail-at-scale amplification** — fan-out makes aggregate latency track component tails.
- **Throughput** — operations completed per unit time.
- **TTL** — time-to-live; validity window of a value.
- **Thread-per-core** — pin one thread per core, partition data, avoid cross-core sync.
- **Utilization (ρ)** — fraction of capacity in use; latency explodes as $\rho \rightarrow 1$.
- **Backpressure strategies** — block / drop / (bounded) buffer.

Companion docs: `DEVELOPER_GUIDE.md` (RVO's concrete design decisions and tradeoffs), `ARCHITECTURE.md` (data flow). This file is the conceptual layer beneath both.

RVO Benchmarking Guide

This document is the end-to-end reference for running, reading, and extending the RVO benchmark suite. It covers both the micro-benchmarks (per-operation service times) and the macro load harness (sustained-throughput, tail-latency, and graceful-degradation curves), plus the plotting pipeline and how to use the numbers in the Stage 3 tech report.

For methodology rationale, measured results, and architectural claims, see [BENCHMARK_PERFORMANCE.md](#).

Table of Contents

1. [Why benchmarking matters here](#)
2. [Suite overview](#)
3. [Environment requirements](#)
4. [Running the micro-benchmarks](#)
5. [Running the macro load harness](#)
6. [Output files — schema and meaning](#)
7. [Generating figures](#)
8. [Interpreting each figure](#)
9. [Statistical rigour checklist](#)
10. [Adding a new scenario](#)
11. [Common pitfalls](#)
12. [Using the numbers in the tech report](#)

1. Why benchmarking matters here

RVO’s core claim is that bounded queues + cost-hint load-shedding + decoupled gRPC inference keep the control-loop tail latency flat even when detectors are slow or the camera is overloaded. Numbers are required to substantiate that claim in any interview, tech report, or paper. Without them the architecture is a design argument, not an evaluated system.

The three specific properties that must be demonstrated:

Property	Measured by
HOL blocking: in-process detector latency appears directly in tick p50 and p99	Fig 1 — tick p50 tracks <code>blocking_*</code> detector sleep ms-for-ms
Load-shedding: High-cost overrunning detector is backed off, tick p99 stays near-baseline	Fig 2 — <code>load_shed</code> time-series: skips rise, tick p99 stays low
Graceful degradation: overload raises frame loss, not latency	Fig 3 — <code>overLoad_*</code> : <code>frame_loss_rate</code> rises with fps, tick p99 stays bounded

Fig 4 (CDF) and Fig 5 (throughput ceiling) provide supporting context.

2. Suite overview

crates/rvo-bench/
 benches/micro.rs
 src/bin/load_harness.rs
 src/lib.rs

← criterion micro-benchmarks (per-op service times)
← macro load harness (sustained run, CSV output)
← HistSummary, CounterSnapshot, CsvWriter shared by harness

target/bench_results/
target/criterion/
docs/report/figures/

← harness output (CSVs); gitignored
← criterion HTML reports; created by cargo bench
← generated figures; gitignored

Micro-benchmarks (criterion) measure the cost of a single operation in isolation: `SignalStore::upsert`, `FrameBuffer::push`, `EventEngine::update`, `Scheduler::tick` with 0/1/4/8 null detectors. These establish the per-operation service times that feed a back-of-envelope capacity model.

Macro load harness drives the full scheduler at configurable fps with configurable detector mixes for a sustained duration (default 30s + 5s warm-up). It captures all histogram percentiles and counter deltas into per-interval time-series CSVs and a final summary CSV.

3. Environment requirements

WSL2 is not valid for p99/p99.9 numbers. The hypervisor scheduler and the lack of CPU isolation make tail latencies meaningless — values can be 10× higher than bare-metal and are not reproducible. Develop and iterate on WSL; run the headline benchmarks on bare-metal Linux or a dedicated VM.

Mandatory before any bench run

```
# 1. Set CPU governor to performance (prevents freq scaling jitter)
sudo cpupower frequency-set -g performance

# 2. Verify governor is set on all cores
cat /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor | sort -u
# expected output: performance

# 3. Disable turbo boost (reduces variance in p99.9)
# Intel:
echo 1 | sudo tee /sys/devices/system/cpu/intel_pstate/no_turbo
# AMD:
echo 0 | sudo tee /sys/devices/system/cpu/cpufreq/boost

# 4. Optional: pin the harness to isolated cores (strongest isolation)
# First isolate cores at boot via isolcpus=2,3 in GRUB, then:
taskset -c 2,3 ./target/release/load_harness --scenario baseline
```

Recommended: document your hardware

Record the following in your tech report's evaluation section before quoting any number:

```
CPU:      <model, GHz, core count>
RAM:      <GB, speed>
OS:       <distro, kernel version>
Rust:     <rustc --version>
Profile:  release, LTO=true, codegen-units=1
Governor: performance, turbo: disabled
```

4. Running the micro-benchmarks

```
# Build and run all micro-benchmarks (release mandatory)
cargo bench -p rvo-bench --bench micro

# Run only the signal_store group
cargo bench -p rvo-bench --bench micro -- signal_store

# Run only the scheduler_tick group
cargo bench -p rvo-bench --bench micro -- scheduler_tick
```

HTML reports land in `target/criterion/`. Open `target/criterion/report/index.html` in a browser for full violin plots and regression history.

What each group measures

Group	Benchmarks	What it tells you
signal_store	upsert, get_hit, get_miss_expired	Cost of the seqlock-protected slot read/write on the hot path
frame_buffer	push_300, slice_window_10s	Cost of ring-buffer append and clip-window slice
event_engine	update_no_fire, update_fires	Cost of temporal state machine evaluation per tick
scheduler_tick	no_detectors, null_detectors/1/4/8	Pure scheduler overhead + linear cost per detector gate check

The scheduler tick numbers are the capacity model anchor: if `null_detectors/8` costs $X \mu\text{s}$, adding 8 real detectors costs at minimum $X \mu\text{s}$ /tick of overhead (detector work is additive).

5. Running the macro load harness

One command: all 13 scenarios, single run

```
cargo build -p rvo-bench --bin load_harness --release
./target/release/load_harness --all
```

This runs all 13 scenarios sequentially (30s measurement + 5s warm-up each), writing each result to `target/bench_results/summary.csv`. Any stale `summary.csv` is removed automatically before the first scenario starts.

Multiple runs for variance (recommended for reports)

```
# 5 repeats of every scenario - produces 65 rows in summary.csv with a run_id column
./target/release/load_harness --all --runs 5

# 3 repeats with 60-second measurement windows (better p99.9 sample count)
./target/release/load_harness --all --runs 3 --duration-secs 60
```

Each repeat appends rows with a distinct `run_id` (1..=N). Group by `scenario` and compute mean ± stddev across `run_id` values to get variance-aware results.

Custom duration / single scenario

```
# 60-second runs (better p99.9 sample count)
./target/release/load_harness --all --duration-secs 60

# Single scenario
./target/release/load_harness --scenario blocking_10ms --duration-secs 30

# Single scenario, custom output dir
./target/release/load_harness --scenario load_shed --out-dir /tmp/bench
```

The 13 scenarios

HOL-BLOCKING GROUP — NO SHEDDING (COST=LOW), DETECTOR RUNS EVERY TICK

`max_fps=10000` (min_interval = 0.1ms) ensures the detector runs on every scheduler tick. `tick_p50` therefore directly measures detector latency — not scheduler overhead. This is the correct metric for HOL blocking: a 10ms detector produces a 10ms tick p50.

Scenario	Detector	tick_p50 expected	Purpose
<code>baseline</code>	none	~5µs	Pure scheduler overhead — the floor
<code>inproc_low</code>	DummyDetector (~0ms)	~5µs	Cheap in-process baseline
<code>blocking_1ms</code>	LatencyDetector(1ms, Low, 10000fps)	~1ms	HOL blocking — mild
<code>blocking_3ms</code>	LatencyDetector(3ms, Low, 10000fps)	~3ms	HOL blocking — moderate
<code>blocking_10ms</code>	LatencyDetector(10ms, Low, 10000fps)	~10ms	HOL blocking — heavy
<code>blocking_50ms</code>	LatencyDetector(50ms, Low, 10000fps)	~50ms	HOL blocking — severe; also causes ring-buffer frame loss

`blocking_50ms` is special: with `tick_p50` ≈ 50ms the scheduler drains at only ~19.7/s, which is below the 30fps camera rate. Ring-buffer overwrites occur (~10 frames/s lost). `frame_loss_rate` in the CSV captures this.

LOAD-SHEDDING GROUP — BACKOFF ACTIVE (COST=HIGH)

Scenario	Detectors	Camera fps	Purpose
<code>load_shed</code>	DummyDetector + LatencyDetector(50ms, High, 60fps)	~30fps	Backoff in action: tick p99 near-baseline

Why 60fps for the slow detector: `budget = (1/60) × 2 = 33ms`. The 50ms sleep exceeds 33ms → overrun fires → `apply_backoff(High)` → 500ms backoff. During backoff, DummyDetector continues running freely. Tick p99 stays near-baseline (~5µs); `total_ticks` >> 5000 in 30s (a slow-only scheduler would reach ~600).

OVERLOAD GROUP — SLOW DETECTOR, CAMERA FPS SWEEPS ABOVE SCHEDULER CEILING

All three use `LatencyDetector(5ms, Low, 1000fps)` (min_interval=1ms, runs every tick):

```
tick cost      = 5ms detector + 0.5ms inter-tick sleep ≈ 5.5ms/tick
effective rate ≈ 176/s

overload_threshold: 120fps camera < 176/s → no frame loss (reference)
overload_moderate:  300fps camera > 176/s → ~120 frames/s lost in ring buffer
overload_severe:    600fps camera > 176/s → ~400 frames/s lost in ring buffer
```

Frame loss is captured by `frame_loss_rate = actual_camera_fps - effective_fps` in the CSV, not by `total_frame_drops` (which stays 0 — the bounded channel never saturates because the scheduler batch-drains all pending frames per tick; ring-buffer overwrites are silent by design).

Scenario	Camera fps	Expected effective_fps	Expected frame_loss_rate
<code>overload_threshold</code>	120fps	~176/s	0
<code>overload_moderate</code>	300fps	~176/s	~124/s
<code>overload_severe</code>	600fps	~176/s	~424/s

THROUGHPUT CEILING GROUP — FAST DETECTOR, CAMERA FPS PROBES SCHEDULER CEILING

DummyDetector tick rate ≈ 1756/s. Scenarios probe below and above that ceiling to show when the ring buffer starts losing frames. `actual_camera_fps` is measured (not assumed) because `thread::sleep` granularity caps the camera thread below its configured rate at high fps.

Scenario	Configured fps	Typical actual fps	Expected outcome
<code>fps_1000</code>	1000fps	~938/s	Below ceiling → 0 frame loss (reference)
<code>fps_2000</code>	2000fps	~1772/s	Near ceiling → marginal loss (~14/s)
<code>fps_5000</code>	5000fps	~3841/s	Above ceiling → heavy loss (~2086/s)

`frame_loss_rate = (actual_camera_fps - effective_fps).max(0)`. If the camera thread cannot reach its configured rate, `frame_loss_rate` correctly reports 0 rather than attributing phantom losses.

What the harness prints during a run

```
[harness] scenario=blocking_10ms duration=30s warmup=5s sample=1000ms
[harness] warming up for 5s ...
[harness] warm-up done, measuring ...
[harness] t=6.0s tick_p99=10.10ms skips/s=0 frame_drops/s=0
...
[harness] DONE tick_p50=10.10ms tick_p99=10.34ms tick_p999=10.39ms ticks=2793 frame_drops=0
[BENCH VALIDATION OK] blocking_10ms: tick_p50=10.10ms ≈ 10ms injected sleep, 0 frame drops
```

For blocking scenarios, `tick_p99` should be close to the detector sleep. For `blocking_50ms` with ring-buffer loss, a NOTE is printed showing the frame loss rate.

6. Output files — schema and meaning

`target/bench_results/summary.csv`

One row per scenario per run. The single source of truth for Figures 1, 3, and 4.

Column	Unit	Meaning
<code>run_id</code>	int	Run number (1..=N from <code>--runs N</code> ; always 1 for single runs)
<code>scenario</code>	string	Scenario name
<code>detector_sleep_ms</code>	ms	Configured detector latency (0 for non-blocking)
<code>input_fps</code>	fps	Configured synthetic camera rate
<code>actual_camera_fps</code>	fps	Measured camera send rate during the measurement window (may be below <code>input_fps</code> at high rates due to <code>thread::sleep</code> granularity)
<code>duration_secs</code>	s	Measurement window (after warm-up)
<code>tick_p50_ns</code>	ns	Median tick duration over the measurement window
<code>tick_p99_ns</code>	ns	99th percentile tick duration
<code>tick_p999_ns</code>	ns	99.9th percentile tick duration
<code>tick_count</code>	count	Total tick samples in the histogram
<code>exec_p50_ns</code>	ns	Median detector execute() duration (all detectors combined)
<code>exec_p99_ns</code>	ns	p99 detector execute() duration
<code>exec_p999_ns</code>	ns	p99.9 detector execute() duration
<code>total_ticks</code>	count	Scheduler ticks fired during measurement
<code>total_execs</code>	count	Detector execute() calls
<code>total_skips</code>	count	Detector gate skips (FPS cap + backoff + disabled)
<code>total_events</code>	count	Events emitted by the event engine
<code>total_frame_drops</code>	count	Frames dropped at the camera channel level (bounded channel full). This stays 0 for overload scenarios because the scheduler batch-drains the channel on every tick; ring-buffer losses are captured in <code>frame_loss_rate</code> instead.
<code>effective_fps</code>	fps	$\text{total_ticks} / \text{duration_secs}$ — the scheduler's actual processing rate
<code>frame_loss_rate</code>	fps	$\max(0, \text{actual_camera_fps} - \text{effective_fps})$ — frames/s lost in the ring buffer because the camera outpaced the scheduler

Reading `frame_loss_rate`: a non-zero value means the scheduler cannot process every frame the camera produces. Frames are silently overwritten in the ring buffer (newest survives). This is expected for `blocking_50ms`, `overload_moderate`, `overload_severe`, `fps_2000`, and `fps_5000`.

Minimum sample count for valid p99.9: the HDR histogram needs ≥ 1000 samples per percentile decade, so $\geq 10,000$ tick samples for a reliable p99.9. At the default ~ 1756 Hz tick ceiling, 30s gives $\sim 50,000$ samples —

sufficient for fast scenarios. For slow scenarios (`blocking_50ms` : ~590 ticks in 30s), p99.9 is the same as p99 (small sample counts compress the tail). Extend `--duration-secs` or do not report p99.9 for slow scenarios.

`target/bench_results/<scenario>_<duration>s_timeseries.csv`

One row per sample interval (default 1s). Source for Figure 2 (load-shedding time-series). When `--runs N` is used, each run overwrites the timeseries file; only the last run's timeseries is kept per scenario. Use `--runs 1` if you need all timeseries files.

Column	Unit	Meaning
<code>elapsed_ms</code>	ms	Wall time since measurement start (0 = end of warm-up)
<code>ticks_delta</code>	count	Ticks fired in this interval
<code>execs_delta</code>	count	execute() calls in this interval
<code>skips_delta</code>	count	Gate skips in this interval — rising = load-shedding active
<code>events_delta</code>	count	Events emitted in this interval
<code>frame_drops_delta</code>	count	Channel-level frame drops in this interval
<code>tick_p50_ns</code>	ns	p50 tick over the full run so far (HDR is cumulative)
<code>tick_p99_ns</code>	ns	p99 tick over the full run so far
<code>exec_p50_ns</code>	ns	p50 detector exec over the full run so far
<code>exec_p99_ns</code>	ns	p99 detector exec
<code>staleness_p50_ns</code>	ns	p50 frame staleness (camera→scheduler latency)
<code>staleness_p99_ns</code>	ns	p99 frame staleness
<code>frame_queue_depth</code>	count	Live frame channel depth at sample time

Note: because the HDR histogram is cumulative (not windowed), p99 in the timeseries represents the p99 of all ticks since warm-up, not just the last interval. Use the time-series primarily for observing `skips_delta` trends; use the summary CSV for end-of-run percentile comparisons.

7. Generating figures

See [PLOT_GUIDE.md](#) for: - The complete CSV column schema - What each of the five figures shows, what axes to use, and what “good data” looks like - A plotting recipe that works in Python, R, or Excel - How to interpret the harness validation output

Five figures are produced:

Figure	File	Claim it supports
Fig 1	<code>fig1_tick_p50_vs_detector_latency.pdf</code>	HOL blocking: tick p50 tracks detector sleep ms-for-ms
Fig 2	<code>fig2_load_shedding.pdf</code>	Backoff keeps tick fast while shedding slow detector
Fig 3	<code>fig3_overload_graceful_degradation.pdf</code>	Frame loss rises under overload; tick p99 stays bounded
Fig 4	<code>fig4_tick_cdf.pdf</code>	Tail latency distribution per scenario
Fig 5	<code>fig5_throughput_ceiling.pdf</code>	Fast pipeline: frame loss appears only above the scheduler tick-rate ceiling

8. Interpreting each figure

Figure 1 — HOL blocking: tick p50 vs detector latency

What it shows: `tick_p50` (and `tick_p99`) on the Y-axis vs configured detector sleep on the X-axis, with a horizontal reference line at the `baseline` (no detectors) value.

What to look for: - `tick_p50` tracks `detector_sleep_ms` linearly and tightly. - `tick_p99` and `tick_p999` are close to p50 (the sleep is deterministic). - The baseline reference line is flat at $\sim 5\mu\text{s}$, well below all blocking curves.

Key claim supported: “an in-process detector that runs synchronously on every tick imposes its full execution time as direct tick latency — there is no buffering.”

Red flags in the data: - `tick_p50` equals the baseline ($\sim 5\mu\text{s}$) for all blocking scenarios — the detector is not running every tick. Check that `max_fps=10000` in `detectors_for()`. - Baseline `tick_p99` > 1ms — system noise. Check governor and other processes.

Figure 2 — Load-shedding: time-series

What it shows: dual-axis time-series for the `load_shed` scenario. Top panel: rolling tick p99. Bottom panel: `skips_delta` (orange bars) and `frame_drops_delta` (red bars) per sample interval.

What to look for: - `skips_delta` is non-zero from the start and is consistently non-zero throughout the run (the 50ms detector is shed on its first execution, then backed off for 500ms windows). - `frame_drops_delta` stays at 0 — the fast path is not overwhelmed. - `tick_p99` stays near-baseline ($\sim 10\mu\text{s}$), not near 50ms. - `total_ticks` in `summary.csv` >> 5000 (fast DummyDetector ticks dominate the count). - `total_execs` ≈ 900 over 30s — DummyDetector running at its 30fps cadence.

Key claim supported: “the backoff mechanism sheds the slow detector without starving the fast path — DummyDetector continues running freely between backoff windows.”

Red flags: - `skips_delta` is zero — backoff never triggered. Verify `cost_hint = High` and `max_fps = 60.0` for the LatencyDetector in `load_shed` (budget = 33ms < 50ms sleep). - `tick_p99` approaches 50ms — load-shedding is not effective. - `total_ticks` ≈ 600 — the scheduler ran at the slow detector’s pace, not the fast one.

Figure 3 — Graceful degradation: overload_* scenarios

What it shows: `frame_loss_rate` (left Y, red) and `tick_p99` (right Y, blue) vs `actual_camera_fps`, for the three overload scenarios.

What to look for: - `overload_threshold` (camera $\approx 118\text{fps}$ < effective 176/s): `frame_loss_rate` = 0. - `overload_moderate` (camera $\approx 294\text{fps}$ > 176/s): `frame_loss_rate` $\approx 118/\text{s}$. - `overload_severe` (camera $\approx 577\text{fps}$ > 176/s): `frame_loss_rate` $\approx 401/\text{s}$. - `tick_p99` stays roughly constant across all three ($\approx 5\text{ms}$, the detector sleep). - `total_frame_drops` is 0 for all three — losses happen in the ring buffer, not the channel.

Key claim supported: “bounded queues degrade gracefully: ring-buffer overwrites absorb excess frames, keeping tick latency predictable under overload.”

Note on validation: the harness validates overload scenarios by checking that `effective_fps < actual_camera_fps` for moderate/severe. If this check fails (e.g. the slow detector is not running every tick), the harness exits 1 with a diagnostic.

Red flags: - `frame_loss_rate` = 0 for `overload_moderate` — effective tick rate $\geq$ camera fps. Check that `max_fps=1000` (not 30) for the overload LatencyDetector. - `tick_p99` grows sharply with fps — something is queuing latency, not dropping frames.

Figure 4 — Tick CDF

What it shows: approximate CDF of tick duration for four scenarios (`baseline`, `inproc_low`, `blocking_3ms`, `blocking_10ms`). X-axis: latency in ms. Y-axis: percentile (50th to 99.9th).

Note on approximation: the CDF is interpolated from three reported percentiles (p50/p99/p99.9). It is an approximation — not a full empirical CDF — because the raw histogram buckets are not exported to CSV. Label it as such in any paper.

What to look for: - Curves separate cleanly: baseline lowest, then `inproc_low`, then `blocking_3ms`, then `blocking_10ms`. - The p50 of each blocking curve aligns with the injected sleep (3ms and 10ms). - The curves are steep (low jitter) — the sleep is deterministic so most ticks cluster tightly around the p50 value.

Figure 5 — Throughput ceiling: fps_* scenarios

What it shows: `frame_loss_rate` (left Y, red) and `tick_p99` (right Y, blue) vs `actual_camera_fps`, for `fps_1000`, `fps_2000`, `fps_5000`.

What to look for: - `fps_1000` (actual $\approx 938\text{fps} < 1756/\text{s}$ ceiling): `frame_loss_rate` = 0. - `fps_2000` (actual $\approx 1772\text{fps} \approx$ ceiling): marginal loss ($\sim 15/\text{s}$). - `fps_5000` (actual $\approx 3841\text{fps} >$ ceiling): heavy loss ($\sim 2086/\text{s}$). - `tick_p99` stays flat across all three — the fast detector does not slow down as camera rate rises.

Key claim: this figure identifies the DummyDetector scheduler’s throughput ceiling ($\sim 1756/\text{s}$) and shows how frame loss scales when the camera exceeds it. Compare with Figure 3: in Figure 3 the ceiling is much lower ($\sim 176/\text{s}$) because the slow detector consumes each tick for 5ms. The paired comparison isolates the detector as the cause of Figure 3’s earlier onset of frame loss.

Note on actual vs configured fps: `actual_camera_fps` is measured per-run. At configured 5000fps the camera thread achieves only $\sim 3841\text{fps}$ due to `thread::sleep` granularity. The `frame_loss_rate` column uses `actual_camera_fps` so the reported loss is accurate (not inflated by a configured rate the camera never reached).

9. Statistical rigour checklist

Before quoting any number in a report or paper:

- ☐ **≥ 5 runs per scenario** on the same machine, same governor setting. Run with `--runs 5`. Report median $\pm$ 95% CI, not a single run.
- ☐ **Warm-up window excluded.** Default is 5s. For slow-converging scenarios (`blocking_50ms`), consider `--warmup-secs 10`.
- ☐ **$\geq 10,000$ tick samples for p99.9.** Check `tick_count` in summary.csv. Fast scenarios at 30s give $\sim 50,000$ samples — fine. `blocking_50ms` produces only ~ 590 ticks in 30s; do not report p99.9 for it or extend `--duration-secs`.
- ☐ **Baseline recorded in the same session.** `--all` runs baseline immediately before other scenarios, keeping hardware state (caches, TLB) consistent.
- ☐ **No other significant load on the machine.** Check with `htop` before starting.
- ☐ **Hardware spec documented.** Every quoted number must name the CPU, RAM, OS kernel, and Rust version.
- ☐ **`actual_camera_fps` vs `input_fps` noted for fps_* scenarios.** The configured fps and the achieved fps diverge at high rates. Quote `actual_camera_fps` in reports, not `input_fps`.

☐ **Coordinated omission acknowledged.** Tick latency is measured from when `scheduler.tick()` is called, not from when the frame was due. Queuing delay at the camera channel is not captured. This is acceptable for a control-loop latency claim but must be noted as a limitation.

Computing mean $\pm$ CI from multi-run output

```
import pandas as pd, scipy.stats as st

df = pd.read_csv("target/bench_results/summary.csv")
results = []
for scenario, group in df.groupby("scenario"):
    tick_p99_ms = group["tick_p99_ns"] / 1e6
    n = len(tick_p99_ms)
    mean = tick_p99_ms.mean()
    sem = tick_p99_ms.sem()
    ci95 = st.t.ppf(0.975, df=n-1) * sem if n > 1 else float("nan")
    results.append({"scenario": scenario, "n": n,
                   "tick_p99_ms_mean": mean, "tick_p99_ms_ci95": ci95,
                   "frame_loss_mean": group["frame_loss_rate"].mean()})
print(pd.DataFrame(results).to_string(index=False))
```

10. Adding a new scenario

1. Add the scenario name and detector list to `detectors_for()` in `load_harness.rs`.
2. If it needs a different camera fps, add it to `camera_fps_for()`.
3. If it has a meaningful `detector_sleep_ms`, add it to the match near the end of `run()`.
4. Add the scenario name to `ALL_SCENARIOS` so `--all` includes it.
5. Add a `validate_scenario()` case if the scenario should self-check that its intended mechanism fired.
6. Update `PLOT_GUIDE.md` with the scenario's axes and expected output.

11. Common pitfalls

`tick_p50` stays at ~5 μ s for blocking scenarios

The detector is not running every tick. Verify that `max_fps=10000` (not 30) for `blocking_1/3/10ms` scenarios in `detectors_for()`. With `max_fps=30`, the detector fires on only ~1.74% of ticks and tick p50 always reports scheduler overhead, hiding the HOL effect.

`frame_loss_rate` is 0 for overload_moderate

The effective tick rate is $\geq$ the actual camera fps. Most likely the overload LatencyDetector has the wrong `max_fps` (should be 1000 so it runs every tick). Harness validation will also emit `[BENCH VALIDATION FAIL]` and exit 1.

`load_shed` shows `total_ticks` $\approx$ 600

Backoff is not activating. Check: 1. The LatencyDetector in `load_shed` must use `DetectorCostHint::High` and `max_fps=60.0`. Budget = $(1/60) \times 2 = 33\text{ms}$. The 50ms sleep must exceed this. 2. `total_skips` in `summary.csv` must be > 0 . Zero skips means the overrun check never fired. 3. Running on WSL2 can cause timing jitter that prevents the 50ms sleep from reliably exceeding 33ms in the scheduler's measurement window. Run on bare-metal.

`summary.csv` accumulates rows across multiple `--all` runs

`--all` removes `summary.csv` before the first scenario starts. Single-scenario runs (`--scenario foo`) do not clear the file — they append. Clear manually before a clean session if needed:

```
rm -f target/bench_results/summary.csv
```

Criterion shows high variance

Usually the CPU governor is not set, or turbo is still enabled. Verify:

```
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor # must print: performance
cat /sys/devices/system/cpu/intel_pstate/no_turbo          # must print: 1
```

12. Using the numbers in the tech report

12.1 Experimental setup subsection

```
Hardware: <CPU, RAM>
OS: <kernel>
Toolchain: Rust 1.XX, LTO=true, codegen-units=1
Governor: performance, turbo disabled
Warm-up: 5s excluded from all reported measurements
Runs: 5 per scenario (run_id 1-5), median reported
Sample count: ≥50,000 tick samples per scenario at default tick rate
```

12.2 Micro-benchmark table

Pull p50 and p99 from criterion HTML reports:

Operation	p50 (ns)	p99 (ns)
SignalStore::upsert	—	—
SignalStore::get (hit)	—	—
FrameBuffer::push	—	—
EventEngine::update	—	—
Scheduler::tick (0 detectors)	—	—
Scheduler::tick (8 null detectors)	—	—

12.3 Key results paragraph (fill in from your run)

With no detectors, tick p50/p99 are X/Y μ s (scheduler overhead floor). Adding a 10ms in-process LatencyDetector raises tick p50 to ~10ms — every tick is delayed by the full detector latency with no buffering (HOL blocking on the synchronous execution path). The `load_shed` scenario runs the same 50ms detector alongside DummyDetector with `cost_hint=High`: after the first overrun the scheduler backs off the slow detector for 500ms windows, and tick p99 returns to ~5 μ s (within Z% of baseline) while skips average W/s. Under camera overload at 294fps, with the scheduler capped at ~176/s by the 5ms detector, the ring buffer loses ~118 frames/s while tick p99 stays within T ms — frame loss absorbs the burst without latency explosion.

12.4 Figures

Embed all five PDFs from `docs/report/figures/` in the LaTeX source. Each figure caption must explicitly name what X and Y axes measure and state the key takeaway. Do not embed raw CSV values in captions — state the claim the figure supports.

12.5 Limitations to state honestly

- The harness imposes a `thread::sleep(500 $\mu$ s)` between ticks, capping tick rate at ~1756/s. This is a harness-imposed ceiling, not the hardware limit. A production scheduler with a monotonic timer and no artificial sleep would achieve a higher rate; do not quote this ceiling as a deployment capacity figure.
- Coordinated omission: latency is measured from tick-start, not from frame arrival. Queuing delay at the camera channel is not captured in `tick_p99`.
- `actual_camera_fps` for `fps_5000` is ~3841fps (not 5000fps) because `thread::sleep` at 200 μ s is at the OS timer resolution limit. The configured fps is an upper bound; the measured `actual_camera_fps` is the scientifically valid number.
- Single-machine, single-process evaluation. Distributed scheduling, NUMA effects, and multiple camera streams are not tested.
- `LatencyDetector` uses `thread::sleep` for artificial latency. Real inference has a non-uniform latency distribution with cache and GPU effects absent from a sleep.

Update the “fill in from your run” placeholders once you have real numbers from your bare-metal run. Run with `--runs 5 --duration-secs 60` for report-quality data.

RVO Benchmark Performance

This document records the methodology behind the benchmark design and the experimental results that support RVO's architectural claims. It is the companion to [BENCHMARKING.md](#), which is the operational reference for running the suite. Read that first.

1. Methodology

1.1 Why tick_p50 is the HOL blocking metric

With `max_fps=10000` (`min_interval=0.1ms`), the `blocking_*` detectors run on every scheduler tick. `tick_p50` therefore directly measures detector latency: 10ms detector → ~10ms tick p50. This is the architecturally correct test for HOL blocking — the median tick is as slow as the synchronous bottleneck.

An earlier design used `max_fps=30`, which caused the detector to fire on only ~1.74% of ticks. `tick_p50` always reported scheduler overhead (~5μs), hiding the HOL effect entirely. The change to `max_fps=10000` makes the claim visible in the headline number.

1.2 Why frame_loss_rate, not total_frame_drops

The scheduler batch-drains the bounded frame channel on every tick:

```
while let Ok(frame) = self.frame_rx.try_recv() {
    buf.push(frame);
}
```

This means the channel never fills — `try_send` always succeeds — so `total_frame_drops` stays 0 even under heavy overload. Frame loss instead occurs silently in the `FrameBuffer` ring buffer: frames that arrive between ticks are overwritten by newer ones before the detector reads them.

`frame_loss_rate = max(0, actual_camera_fps - effective_fps)` captures this correctly: if the camera sends frames faster than the scheduler ticks, the difference is frames lost per second in the ring buffer.

1.3 Why actual_camera_fps is measured, not assumed

At high configured fps (2000, 5000), `thread::sleep` granularity caps the camera thread below its target. A configured 5000fps camera thread achieves ~3841fps in practice due to OS timer resolution (~200μs sleep limit). If `frame_loss_rate` used the configured fps, it would report phantom losses for scenarios where the camera never reached the configured rate. Counting actual frames sent during the measurement window and dividing by duration gives an accurate rate that correctly computes `frame_loss_rate = 0` when the scheduler is actually faster than the camera.

1.4 Why the throughput ceiling group uses fps_1000/2000/5000

The `DummyDetector` scheduler ticks at ~1756/s. The `fps_*` group probes around that ceiling: one scenario below it (`fps_1000`, actual ~938fps), one near it (`fps_2000`, actual ~1772fps), one well above it (`fps_5000`, actual ~3841fps). This shows where frame loss begins and how it scales. The group is the paired control for the overload group: at `fps_5000` the fast-detector scheduler still manages its ceiling (1756/s), while the slow-detector overload scenarios cap at only ~176/s — isolating the slow detector as the cause of the earlier overload onset.

1.5 Why overload uses Low cost, not High

`LatencyDetector(5ms, Low, 1000fps)` in the overload group is `Low` cost deliberately. `Low` cost is never backed off by the scheduler. If it were `High`, the backoff mechanism would shed it — which is exactly what `load_shed` demonstrates. The overload group wants a slow tick on every call to cap the scheduler at ~176/s so the camera can outpace it. The `load_shed` group wants the same slow detector to be shed so the fast path stays free. Different cost hints, different outcomes, different claims.

1.6 Multi-run design

Each run appends a row with a distinct `run_id` to `summary.csv`. Grouping by `scenario` and computing median across run IDs gives variance-aware results without manual file management. `--runs 5` is the recommended setting for report-quality data.

2. Measured Results

Results below are from a single representative 30-second run per scenario on this machine. For variance data, run `./target/release/load_harness --all --runs 5` and compute per the snippet in [BENCHMARKING.md §9](#).

2.1 Scheduler overhead (baseline)

metric	value
tick_p50	4.8µs
tick_p99	11.3µs
tick_p999	26.0µs
tick rate	1756/s
frame_drops	0

The ~5µs p50 is the floor: lock acquisition on the frame buffer, ring-buffer push, HDR histogram record, and loop overhead. Everything above this in other scenarios is attributable to detector cost.

2.2 HOL blocking

scenario	tick_p50	tick_p99	effective_fps	frame_loss_rate
inproc_low (DummyDetector)	4.8µs	11.4µs	1759/s	0
blocking_1ms	1.07ms	1.15ms	610/s	0
blocking_3ms	3.07ms	3.16ms	275/s	0
blocking_10ms	10.10ms	10.34ms	93/s	0
blocking_50ms	50.20ms	50.36ms	19.7/s	10.2/s

`tick_p50` tracks the injected sleep ms-for-ms with low jitter (p50≈p99 because the sleep is deterministic). `blocking_50ms` is the only one where frame loss occurs — the 50ms tick interval puts the drain rate (19.7/s) below the 30fps camera rate.

2.3 Load-shedding

metric	value
tick_p50	4.9µs (near-baseline)
tick_p99	13.8µs
tick_p999	50.1ms (rare: first execution before backoff fires)
total_ticks (30s)	47,343
total_execs	900 (DummyDetector at 30fps cadence)
total_skips	93,786 (slow detector shed during 500ms backoff windows)
frame_loss_rate	0

The slow detector fires once, exceeds its 33ms budget (50ms > 33ms), and is parked for 500ms. DummyDetector continues at ~30fps. The tick_p50 stays within 2.5% of baseline. `total_skips` (93,786) >> `total_execs` (900) confirms the slow detector is shed, not running. The occasional 50ms in tick_p999 is the first execution of the slow detector each time its backoff expires — expected behaviour.

2.4 Overload (graceful degradation)

scenario	actual_camera_fps	effective_fps	frame_loss_rate	tick_p99
overload_threshold (ref)	118fps	176/s	0	5.30ms
overload_moderate	294fps	176/s	117.5/s	5.21ms
overload_severe	577fps	176/s	401.1/s	5.22ms

Tick p99 is stable across all three (5.2–5.3ms, equal to the 5ms sleep + overhead). Frame loss scales with the excess camera rate above the 176/s scheduler ceiling. `total_frame_drops` = 0 for all three — the channel never saturates.

2.5 Throughput ceiling

scenario	actual_camera_fps	effective_fps	frame_loss_rate	tick_p99
fps_1000	938fps	1757/s	0	12.1μs
fps_2000	1772fps	1758/s	14.7/s	13.0μs
fps_5000	3841fps	1755/s	2086/s	13.7μs

The DummyDetector scheduler ceiling is ~1756/s. At fps_1000 (actual 938fps) the scheduler is faster than the camera — no loss. At fps_2000 (actual 1772fps) the camera marginally exceeds the ceiling — minimal loss. At fps_5000 (actual 3841fps) the ceiling is significantly exceeded — heavy loss. Tick p99 stays flat (~13μs) because the DummyDetector imposes no latency.

Contrast with the overload group: the same 3841fps would produce ~3665 frames/s loss against the overload ceiling of ~176/s. The slow detector is the difference — it lowers the ceiling by 10×, not the fps itself.

3. Architectural Claims Confirmed

Claim	Evidence
HOL blocking: synchronous detector latency appears directly in tick_p50	blocking_* scenarios: tick_p50 = detector_sleep_ms ± <5%
Load-shedding: High-cost detector shed, fast path unaffected	load_shed: tick_p50 near-baseline, 93,786 skips vs 900 execs
Graceful degradation: overload raises frame_loss_rate, not tick latency	overload_moderate/severe: frame_loss_rate rises, tick_p99 flat at 5.2ms
Throughput ceiling: frame loss begins at actual_camera_fps > scheduler tick rate	fps_1000: 0 loss; fps_2000: 14.7/s loss; fps_5000: 2086/s loss
Panic isolation: panicking detector disabled, pipeline survives	unit test: <code>panicking_detector_does_not_kill_scheduler</code>
Bounded clip threads: event burst does not spawn unbounded threads	unit test: <code>event_burst_is_bounded</code>
Non-blocking gRPC path: scheduler tick does not block on network	integration test: <code>grpc_pipeline</code>
Signal TTL: stale signals do not trigger events	unit test: <code>expired_signal_is_absent</code>

4. Limitations

Coordinated omission in tick latency

Tick latency is measured from the moment `scheduler.tick()` is called, not from when the frame was due. Under overload, frames accumulate in the ring buffer between ticks; the queuing delay is not captured in `tick_p99`. The headline latency figure is the control-loop processing time, not the end-to-end camera-to-signal latency. Frame staleness (`staleness_p50_ns` in the timeseries CSV) measures the latter separately.

thread::sleep is not a real model

`LatencyDetector` injects deterministic latency via `thread::sleep`. Real inference has non-uniform latency (GPU sync, CUDA malloc, cold-start effects, memory pressure). The blocking and load-shedding experiments are controlled proxies that isolate the scheduling mechanism, not production model measurements.

Harness-imposed tick ceiling

A `thread::sleep(500µs)` between ticks caps the scheduler at ~1756/s. A production deployment using a monotonic timer without artificial sleep would achieve a higher throughput ceiling. The ~1756/s figure characterises the harness, not the hardware limit.

actual_camera_fps at high configured rates

`thread::sleep` granularity limits the camera thread. Configured 5000fps produces only ~3841fps actual. The `frame_loss_rate` column uses `actual_camera_fps` to remain accurate, but the configured rate cannot be independently verified without a spin-loop camera (which would consume a full CPU core and distort the benchmark).

Single-machine, single-process

All results are from one machine. Distributed scheduling, NUMA effects, multiple concurrent camera streams, and gRPC network jitter are not covered by the load harness. The gRPC path is exercised separately by `grpc_pipeline` integration tests.

Benchmark Plot Guide

This document describes the five benchmark figures that the RVO bench suite produces, what each one proves, and how to recreate them from the CSV files that `load_harness --all` generates.

`scripts/plot.py` is the local plotting script used for paper and report writing. It is not tracked in the repository. You can recreate it in any tool you prefer (Python/matplotlib, R, Excel, Observable) — this document gives you everything you need: CSV schema, axes, and the claim each figure must support.

Running the bench suite

```
# on bare-metal Linux with performance governor set
cargo build -p rvo-bench --bin load_harness --release

# Single run - 30s per scenario
./target/release/load_harness --all

# 5 runs for variance (recommended for reports)
./target/release/load_harness --all --runs 5

# 60s measurement windows for better p99.9 sample counts
./target/release/load_harness --all --runs 3 --duration-secs 60
```

All output lands in `target/bench_results/` (gitignored). Two file types:

- `summary.csv` — one row per scenario per run, end-of-run aggregates
- `<scenario>_<duration>s_timeseries.csv` — per-second samples during the run

When using `--runs N`, each run overwrites the timeseries file (only the last run's timeseries is retained per scenario). The summary.csv accumulates all N rows.

CSV schema

`summary.csv`

Column	Unit	Description
<code>run_id</code>	int	Run number (1..=N from <code>--runs N</code> ; always 1 for single runs)
<code>scenario</code>	string	Scenario name
<code>detector_sleep_ms</code>	ms	Configured detector artificial latency (0 for non-blocking)
<code>input_fps</code>	fps	Configured synthetic camera rate
<code>actual_camera_fps</code>	fps	Measured camera send rate during the measurement window
<code>duration_secs</code>	s	Measurement window (after warm-up)
<code>tick_p50_ns</code>	ns	Median tick duration
<code>tick_p99_ns</code>	ns	99th-percentile tick duration
<code>tick_p999_ns</code>	ns	99.9th-percentile tick duration
<code>tick_count</code>	count	Total tick samples in histogram
<code>exec_p50_ns</code>	ns	Median detector <code>execute()</code> duration
<code>exec_p99_ns</code>	ns	p99 detector <code>execute()</code> duration
<code>exec_p999_ns</code>	ns	p99.9 detector <code>execute()</code> duration
<code>total_ticks</code>	count	Scheduler ticks during measurement
<code>total_execs</code>	count	Detector <code>execute()</code> calls
<code>total_skips</code>	count	Detector gate skips (FPS cap + backoff + disabled)
<code>total_events</code>	count	Events emitted by the event engine
<code>total_frame_drops</code>	count	Channel-level frame drops (bounded channel full). Stays 0 for overload/fps_* scenarios because the scheduler batch-drains the channel every tick.
<code>effective_fps</code>	fps	$\text{total_ticks} / \text{duration_secs}$ — scheduler's actual processing rate
<code>frame_loss_rate</code>	fps	$\max(0, \text{actual_camera_fps} - \text{effective_fps})$ — frames/s lost in ring buffer

Important: for overload and fps_* scenarios, use `frame_loss_rate` (not `total_frame_drops`) to assess frame loss. The bounded channel never saturates because the scheduler drains it completely on every tick; ring-buffer overwrites are silent.

***_timeseries.csv**

Column	Unit	Description
<code>elapsed_ms</code>	ms	Wall time since measurement start (0 = end of warm-up)
<code>ticks_delta</code>	count	Ticks in this sample interval
<code>execs_delta</code>	count	<code>execute()</code> calls in this interval
<code>skips_delta</code>	count	Gate skips in this interval
<code>events_delta</code>	count	Events emitted in this interval
<code>frame_drops_delta</code>	count	Channel-level frame drops in this interval
<code>tick_p50_ns</code>	ns	p50 cumulative tick latency (since warmup end)
<code>tick_p99_ns</code>	ns	p99 cumulative tick latency
<code>exec_p50_ns</code>	ns	p50 cumulative detector exec latency
<code>exec_p99_ns</code>	ns	p99 cumulative detector exec latency
<code>staleness_p50_ns</code>	ns	p50 frame staleness (camera→scheduler)
<code>staleness_p99_ns</code>	ns	p99 frame staleness
<code>frame_queue_depth</code>	count	Live frame channel depth at sample time

The five figures

Figure 1 — HOL blocking: tick p50 vs detector sleep

Source: `summary.csv`, rows where `scenario` starts with `blocking_`

Axes: - X: `detector_sleep_ms` - Y (primary): `tick_p50_ns / 1e6` (ms) - Y (secondary): `tick_p99_ns / 1e6` (ms) - Reference line: `tick_p50_ns / 1e6` from the `baseline` row (~0.005ms)

Claim: With `max_fps=10000` (`min_interval=0.1ms`), the detector runs on every scheduler tick. `tick_p50` therefore directly tracks detector latency: a 10ms detector produces a ~10ms tick p50. This is the correct metric for demonstrating HOL blocking — the median tick is as slow as the slowest synchronous step.

What good data looks like: - `tick_p50_ns / 1e6` closely tracks `detector_sleep_ms` (linear, near-unity slope). - `tick_p99_ns` is slightly above `tick_p99_ns` (OS jitter in the tail, not systematic). - The baseline reference line is at ~0.005ms — well below all blocking curves. - `blocking_50ms` shows `effective_fps` ≈ 19.7/s and `frame_loss_rate` ≈ 10/s (tick rate fell below the 30fps camera rate, causing ring-buffer overwrites).

Red flag: `tick_p50_ns` ≈ baseline for all blocking scenarios → detector is not running every tick. Check that `max_fps=10000` (not 30) in `detectors_for()`.

Figure 2 — Load-shedding: time-series

Source: `load_shed_<duration>s_timeseries.csv`

Axes (dual panel): - Top: X = `elapsed_ms / 1000` (s), Y = `tick_p99_ns / 1e6` (ms) - Bottom: X = `elapsed_ms / 1000` (s), bars for `skips_delta` (orange) and `frame_drops_delta` (red)

Claim: The `High`-cost 50ms LatencyDetector exceeds its 33ms budget ($1/60\text{fps} \times 2$) and is backed off for 500ms windows. DummyDetector continues running freely. Tick p99 stays near-baseline despite the 50ms detector existing in the pipeline.

What good data looks like: - `tick_p99_ns / 1e6` stays in the μs range (not approaching 50ms). - `skips_delta` is non-zero every interval (slow detector skipped during backoff windows). - `frame_drops_delta` is 0 (fast path keeps up; load is shed, not queued). - `total_ticks` in `summary.csv` >> 5000 for a 30s run (~47,000 typical). - `total_execs` ≈ 900 (DummyDetector at its 30fps cadence × 30s).

Red flag: `tick_p99_ns` ≈ 50ms and `total_ticks` ≈ 600 → shedding not working. Check `cost_hint=High` and `max_fps=60.0` for the LatencyDetector in `load_shed`.

Figure 3 — Graceful degradation: overload raises frame_loss_rate, tick p99 stays bounded

Source: `summary.csv`, rows where `scenario` starts with `overload_`

Axes (dual Y): - X: `actual_camera_fps` - Left Y: `frame_loss_rate` (fps, red) — NOT `total_frame_drops` - Right Y: `tick_p99_ns / 1e6` (ms, blue) - Rows: `overload_threshold`, `overload_moderate`, `overload_severe`

Claim: A 5ms detector (`max_fps=1000`) caps the scheduler at ~176/s. Cameras above this rate produce frames faster than the scheduler can process them; excess frames are overwritten in the ring buffer. `tick_p99` stays flat — drops absorb the overload.

What good data looks like: - `overload_threshold` (~118fps actual < 176/s): `frame_loss_rate` = 0. - `overload_moderate` (~294fps actual > 176/s): `frame_loss_rate` ≈ 118/s. - `overload_severe` (~577fps actual > 176/s): `frame_loss_rate` ≈ 401/s. - `tick_p99_ns / 1e6` ≈ 5ms for all three rows (the detector sleep). - `total_frame_drops` = 0 for all three (channel never saturates).

Plot note: use `actual_camera_fps` on the X-axis, not `input_fps`. The camera thread may not achieve the configured rate at high fps due to `thread::sleep` granularity.

Red flag: `frame_loss_rate` = 0 for `overload_moderate` → effective tick rate ≥ camera fps. Check `max_fps=1000` for the LatencyDetector in overload scenarios. Harness validation also exits 1 with a diagnostic in this case.

Figure 4 — Tick CDF: tail latency distribution

Source: `summary.csv`, rows for `baseline`, `inproc_low`, `blocking_3ms`, `blocking_10ms`

Axes: - X: interpolated tick latency (ms) using the three percentile points (p50/p99/p99.9) - Y: percentile (50th–99.9th)

Note: this is an approximation from three percentile points, not a full empirical CDF. Label it clearly in any paper.

Claim: The CDF curves separate cleanly by detector sleep. With `max_fps=10000`, the curves are steep — the sleep is deterministic so ticks cluster tightly around p50.

What good data looks like: - `baseline` and `inproc_low` curves nearly overlap (DummyDetector adds ~0μs overhead at baseline tick rates), shifted far left. - `blocking_3ms` and `blocking_10ms` curves are shifted right by exactly their sleep time relative to baseline. - Curves are steep (small spread between p50 and p99.9) — deterministic sleep has low jitter.

Figure 5 — Throughput ceiling: fps_* scenarios

Source: `summary.csv`, rows where `scenario` starts with `fps_`

Axes (dual Y): - X: `actual_camera_fps` (use this, not `input_fps`) - Left Y: `frame_loss_rate` (fps, red) - Right Y: `tick_p99_ns / 1e6` (ms, blue) - Rows: `fps_1000`, `fps_2000`, `fps_5000`

Claim: With DummyDetector, the scheduler ticks at ~1756/s. Below this ceiling (`fps_1000`, actual ≈ 938fps) there is no frame loss. At and above the ceiling (`fps_2000` actual ≈ 1772fps, `fps_5000` actual ≈ 3841fps), loss scales with the excess. This is the fast-path control experiment for Figure 3: it shows that the scheduler's tick rate (not the fps itself) is the limiting factor, and that a fast detector eliminates the early overload seen in Figure 3.

What good data looks like: - `fps_1000`: `frame_loss_rate` = 0 (scheduler is faster than camera). - `fps_2000`: small positive `frame_loss_rate` (actual camera ≈ 1772fps is just above the ~1756/s scheduler ceiling). - `fps_5000`: `frame_loss_rate` ≈ 2086/s (actual camera ≈ 3841fps, scheduler at 1755/s). - `tick_p99_ns` stays flat across all three — the fast detector is not the bottleneck.

Contrast with Figure 3: in Figure 3, frame loss begins at ~176fps effective rate (because the 5ms detector slows every tick). In Figure 5, frame loss begins only at ~1756fps (because DummyDetector imposes no tick overhead). This paired comparison isolates the slow detector as the cause of the early overload in Figure 3.

Plotting recipe (any tool)

All figures follow the same data flow:

1. Load `target/bench_results/summary.csv`.
2. If using `--runs N`, aggregate across `run_id`: compute median (or mean) per `scenario`.
3. Filter rows by `scenario` prefix.

4. Convert `*_ns` columns to ms: divide by `1e6`.
5. Plot the columns listed in each figure's schema above.

For Figure 2 (time-series), load `load_shed<duration>s_timeseries.csv` and plot `elapsed_ms / 1000` on the X-axis.

```
import pandas as pd

df = pd.read_csv("target/bench_results/summary.csv")
df["tick_p50_ms"] = df["tick_p50_ns"] / 1e6
df["tick_p99_ms"] = df["tick_p99_ns"] / 1e6
df["tick_p999_ms"] = df["tick_p999_ns"] / 1e6

# Aggregate across runs (if --runs N was used)
agg = df.groupby("scenario").agg(
    tick_p50_ms_median=("tick_p50_ms", "median"),
    tick_p99_ms_median=("tick_p99_ms", "median"),
    frame_loss_rate_median=("frame_loss_rate", "median"),
    actual_camera_fps_median=("actual_camera_fps", "median"),
).reset_index()
```

In R:

```
df <- read.csv("target/bench_results/summary.csv")
df$tick_p50_ms <- df$tick_p50_ns / 1e6
df$tick_p99_ms <- df$tick_p99_ns / 1e6
agg <- aggregate(cbind(tick_p50_ms, tick_p99_ms, frame_loss_rate, actual_camera_fps)
  ~ scenario, data=df, FUN=median)
```

In Excel: open the CSV, add computed columns `=tick_p50_ns/1000000`, etc.

Interpreting validation output

The harness prints a `[BENCH VALIDATION OK/WARN/NOTE/FAIL]` line after each scenario. `FAIL` exits 1 and stops the run. Examples:

```
[BENCH VALIDATION OK] blocking_10ms: tick_p50=10.10ms ≈ 10ms injected sleep, 0 frame drops
[BENCH VALIDATION NOTE] blocking_50ms: effective 19.7/s < camera 30fps
    (~10 frames/s lost in ring buffer – expected for 50ms HOL scenario). tick_p99=50.36ms
[BENCH VALIDATION OK] load_shed: 47343 ticks, tick_p99=0.01ms (backoff active, fast detector running
freely)
[BENCH VALIDATION OK] overload_moderate: effective 176.5/s < camera 294.0fps
    (~118 frames/s lost in ring buffer, as expected)

[BENCH VALIDATION FAIL] load_shed: 612 ticks recorded (expected >> 5000).
    tick_p99=49.87ms. Load-shedding did not activate – check cost_hint and overrun budget.
[BENCH VALIDATION FAIL] overload_moderate: effective tick rate 1756.0/s ≥ camera 294.0fps
    – scheduler is keeping up, no frame loss. Check that the slow detector runs on every tick.
```

A `FAIL` means the numbers exist in the CSV but do not prove the intended claim. Do not quote those numbers in a report without investigating the root cause.

A `WARN` means a reference scenario produced unexpected data (e.g. frame drops on a fast-path scenario). Investigate but the run continues.

A `NOTE` is informational — the scenario behaved as expected but the outcome involves trade-offs worth noting (e.g. `blocking_50ms` causing ring-buffer loss).